

GUIDE COMPLET DU SYSTÈME DE RECOMMANDATION DE FILMS

De Débutant à Maître — Le Parcours A à Z pour Réussir votre Projet Data Science

Auteur : Guide d'accompagnement pour le projet MovieReco

Niveau : Débutant à Intermédiaire

Objectif : Comprendre et maîtriser chaque brique technologique d'un projet Data Science complet

Projet réel : Système de recommandation de films avec API, Base de données, Frontend et Machine Learning

Table des Matières

1. [Introduction au Projet](#)
 2. [Feuille de Route : Plan d'Action Pas à Pas](#)
 3. [Python pour la Data Science — Les Fondamentaux](#)
 4. [NumPy — Le Cœur du Calcul Scientifique](#)
 5. [Pandas — La Manipulation de Données](#)
 6. [SciPy — Les Mathématiques Avancées](#)
 7. [Scikit-learn — Le Machine Learning](#)
 8. [PostgreSQL — La Base de Données](#)
 9. [SQLAlchemy — L'ORM Python](#)
 10. [FastAPI — L'API Web](#)
 11. [JWT & Authentification — La Sécurité](#)
 12. [Pydantic — La Validation de Données](#)
 13. [Docker & Docker Compose — La Conteneurisation](#)
 14. [Next.js — Le Frontend](#)
 15. [Tailwind CSS — Le Design](#)
 16. [Le Système de Recommandation — Théorie & Pratique](#)
 17. [Guide de Déploiement Pas à Pas](#)
 18. [Ressources et Prochaines Étapes](#)
-

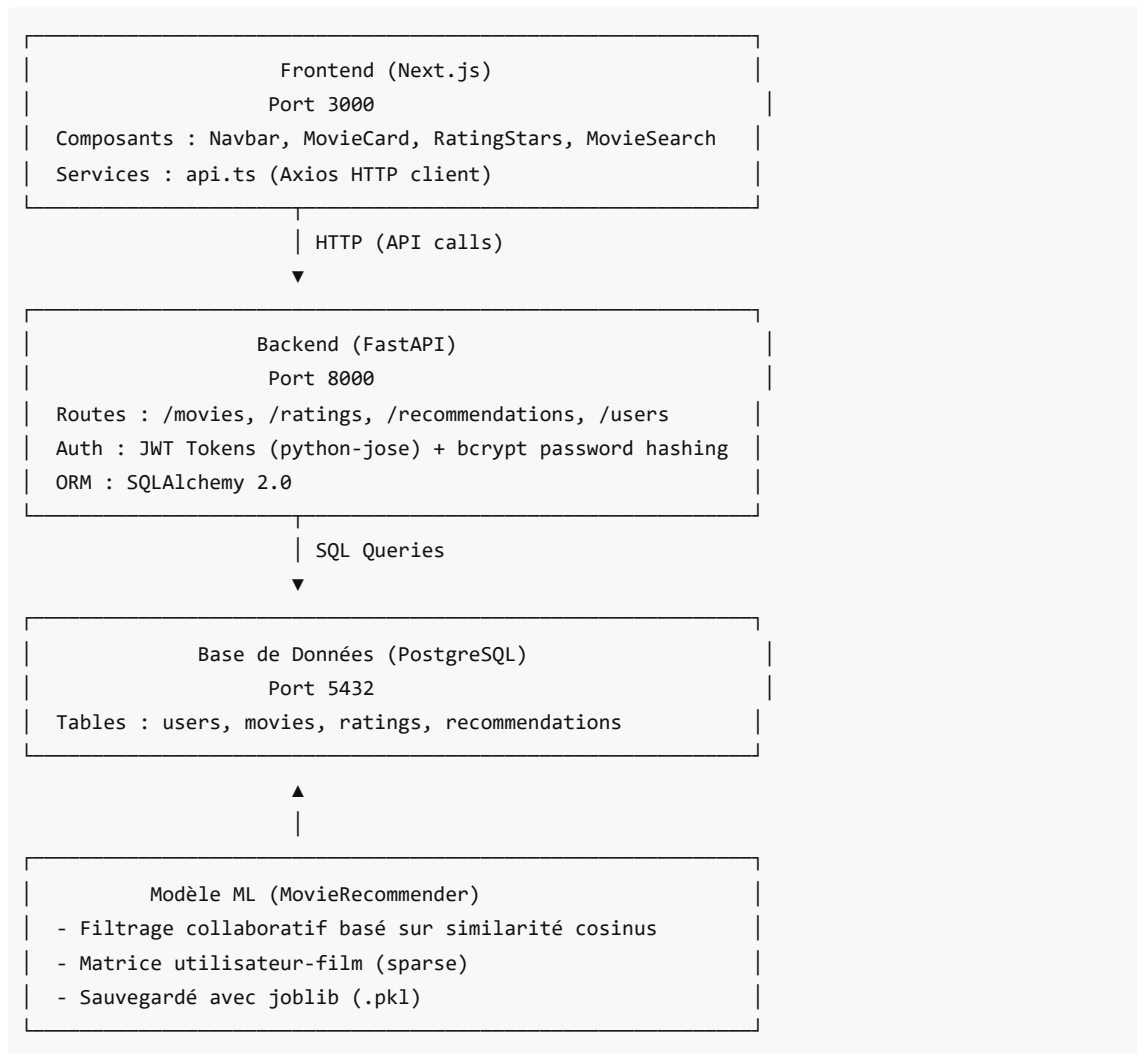
1. Introduction au Projet

1.1 Qu'est-ce que ce projet ?

Ce projet est un **système de recommandation de films** complet, qui combine :

- **Data Science & Machine Learning** : Un algorithme de filtrage collaboratif qui prédit les films qu'un utilisateur pourrait aimer
- **Backend** : Une API REST construite avec FastAPI qui sert les données et les prédictions
- **Base de données** : PostgreSQL pour stocker utilisateurs, films, évaluations et recommandations
- **Frontend** : Une interface web moderne avec Next.js et Tailwind CSS
- **Infrastructure** : Docker pour la conteneurisation et le déploiement

1.2 Architecture du projet



1.3 Les technologies utilisées (et pourquoi)

Technologie	Rôle	Pourquoi ce choix
Python	Langage principal	Standard en Data Science, riche écosystème
FastAPI	API Backend	Performant, moderne, documentation auto-générée
PostgreSQL	Base de données	Robuste, fiable, supporte les requêtes complexes
SQLAlchemy	ORM	Standard Python pour interagir avec la BDD
Pandas	Manipulation de données	Outil #1 pour l'analyse de données tabulaires
NumPy	Calcul numérique	Base de tout le calcul scientifique Python
Scikit-learn	Machine Learning	Bibliothèque ML #1, simple et puissante
SciPy	Maths avancées	Matrices creuses, algèbre linéaire

models/	# Modèles entraînés (.pkl)
docker-compose.yml	# Orchestration Docker
README.md	# Documentation du projet

2. Feuille de Route : Plan d'Action Pas à Pas

 **Cette section est votre plan de bataille.** Chaque étape est numérotée dans l'ordre chronologique. Suivez-les une par une pour réussir le projet. Pour chaque étape, vous saurez :

- **QUOI** : Ce qu'il faut faire
- **QUI** : Quel fichier ou rôle est concerné
- **COMMENT** : Les commandes exactes à exécuter
- **POURQUOI** : L'objectif de l'étape
- **TEMPS** : Estimation du temps nécessaire

Phase 0 : Préparation de l'environnement (Jour 1)

Étape 0.1 — Installer les outils nécessaires

Outil	Version	Lien de téléchargement	Commande de vérification
Python	3.10+	python.org	python --version
Node.js	20+	nodejs.org	node --version
Docker Desktop	Dernière	docker.com	docker --version
Git	Dernière	git-scm.com	git --version
VS Code	Dernière	code.visualstudio.com	code --version
pgAdmin	Dernière	Inclus avec PostgreSQL	Via le menu Démarrer

```
# Vérifications après installation
python --version    # Doit afficher Python 3.10.x ou plus
node --version      # Doit afficher v20.x.x ou plus
npm --version       # Doit afficher 10.x.x ou plus
docker --version    # Doit afficher Docker version xx.x.x
git --version       # Doit afficher git version xx.x.x
```

 **Temps estimé** : 30-60 minutes

Étape 0.2 — Cloner le projet et comprendre sa structure

```
# Si le projet est sur GitHub
git clone https://github.com/votre-compte/movie-recommender.git
cd DATA-SCIENCE

# OU si vous partez de zéro
mkdir DATA-SCIENCE
```

```
cd DATA-SCIENCE
git init

# Explorer la structure
dir          # Windows : liste les fichiers
ls -la       # Mac/Linux : liste les fichiers
```

📁 **Fichiers à examiner :** `README.md` , `docker-compose.yml` , `backend/` , `frontend/`

🕒 **Temps estimé :** 15 minutes

Étape 0.3 — Créer le fichier de configuration (.env)

QUOI : Créer un fichier `.env` à la racine avec les variables d'environnement

COMMENT :

```
# À la racine du projet, créer .env
# GÉNÉRER une clé secrète (ne JAMAIS utiliser celle-ci en production !)
# Sur Mac/Linux :
openssl rand -hex 32 > .env

# Sur Windows PowerShell :
# [System.Text.Encoding]::UTF8.GetString((1..32|'%(Get-Random -Max 256)')) > .env

# OU éditer .env manuellement avec :
echo "JWT_SECRET_KEY=ma-cle-secrete-tres-longue-et-aleatoire" > .env
echo "DATABASE_URL=postgresql://postgres:postgres@localhost:5432/movie_recommender" >> .env
```

⚠ **ATTENTION :** Le fichier `.env` contient des secrets. Ne JAMAIS le commiter dans Git ! Vérifiez qu'il est bien dans `.gitignore` .

🕒 **Temps estimé :** 5 minutes

Phase 1 : La Science des Données (Jours 1-3)

RÔLE : Data Scientist Junior **OBJECTIF :** Comprendre les données, les nettoyer, et construire le modèle ML

FICHIERS CONCERNÉS : `notebooks/01_importation.ipynb` à `notebooks/06_evaluation.ipynb` ,
`backend/recommender.py`

Étape 1.1 — Télécharger le dataset MovieLens

QUOI : Obtenir les données d'entraînement

COMMENT :

```
# 1. Créer le dossier data
mkdir -p data/ml-latest-small

# 2. Télécharger le dataset MovieLens (25 MB)
# Aller sur : https://grouplens.org/datasets/movielens/latest/
# OU directement :
```

```
# Lien direct : https://files.grouplens.org/datasets/movielens/ml-latest-small.zip

# 3. Extraire dans data/ml-latest-small/
# Les fichiers importants sont : movies.csv et ratings.csv
```

VÉRIFICATION :

```
dir data\ml-latest-small\
# Doit contenir : movies.csv, ratings.csv, tags.csv, links.csv
```

 **Temps estimé :** 10 minutes

Étape 1.2 — Notebook d'Importation (EXPLORER les données)

QUOI : Charger et explorer les données dans un notebook Jupyter

QUI : notebooks/01_importation.ipynb

COMMENT :

```
# Dans le notebook 01_importation.ipynb
import pandas as pd
import numpy as np

# Charger les données
movies = pd.read_csv("../data/ml-latest-small/movies.csv")
ratings = pd.read_csv("../data/ml-latest-small/ratings.csv")

# Explorer
print("=== Films ===")
print(f"Nombre de films : {len(movies)}")
print(f"Colonnes : {movies.columns.tolist()}")
print(movies.head())

print("\n=== Évaluations ===")
print(f"Nombre d'évaluations : {len(ratings)}")
print(f"Nombre d'utilisateurs : {ratings['userId'].nunique()}")
print(f"Nombre de films évalués : {ratings['movieId'].nunique()}")
print(f>Note moyenne : {ratings['rating'].mean():.2f}")
print(f"Période : {ratings['timestamp'].min()} à {ratings['timestamp'].max()}")
```

CE QU'IL FAUT COMPRENDRE :

- Le dataset contient ~100 000 évaluations de ~700 utilisateurs sur ~9 000 films
- Chaque évaluation a une note de 0.5 à 5.0 (pas de 1 à 5 classique)
- Les genres sont séparés par | (ex: "Sci-Fi|Action|Thriller")
- La colonne `timestamp` peut être ignorée pour ce projet

 **Temps estimé :** 20 minutes

Étape 1.3 — Notebook de Nettoyage (PROPRETÉ des données)

QUOI : Nettoyer les données : valeurs manquantes, doublons, anomalies

QUI : notebooks/02_nettoyage.ipynb

COMMENT :

```
# 1. Vérifier les valeurs manquantes
print(ratings.isnull().sum())
print(movies.isnull().sum())

# 2. Vérifier les doublons
print(f"Doublons dans ratings : {ratings.duplicated().sum()}")
print(f"Doublons dans movies : {movies.duplicated().sum()}")

# 3. Vérifier les plages de notes
print(f"Notes min/max : {ratings['rating'].min()} / {ratings['rating'].max()}")
assert ratings['rating'].between(0.5, 5.0).all(), "Notes hors limite !"

# 4. Analyser les genres
movies['genre_list'] = movies['genres'].str.split('|')
all_genres = [g for sublist in movies['genre_list'] for g in sublist]
print(f"Genres disponibles : {sorted(set(all_genres))}")

# 5. Extraire l'année du titre
movies['year'] = movies['title'].str.extract(r'\((\d{4})\)')
print(f"Années : {movies['year'].min()} à {movies['year'].max()}")
```

CE QU'IL FAUT RETENIR :

- Les données MovieLens sont déjà assez propres (peu de valeurs manquantes)
- Le principal travail est de comprendre la structure
- Les genres sont stockés avec des pipe | comme séparateur

🕒 **Temps estimé :** 30 minutes

Étape 1.4 — Notebook d'Analyse Exploratoire (COMPRENDRE les données)

QUOI : Analyser les distributions et les tendances

QUI : notebooks/03_analyse.ipynb

COMMENT :

```
import matplotlib.pyplot as plt

# 1. Distribution des notes
ratings['rating'].hist(bins=10)
plt.title("Distribution des notes")
plt.xlabel("Note")
plt.ylabel("Nombre")
plt.show()

# 2. Top 10 des films les plus notés
```

```

movie_stats = ratings.groupby('movieId').agg(
    note_moyenne=('rating', 'mean'),
    nb_notes=('rating', 'count')
).sort_values('nb_notes', ascending=False).head(10)

print("Films les plus notés :")
print(movie_stats.merge(movies, on='movieId'))

# 3. Distribution du nombre de notes par utilisateur
user_stats = ratings.groupby('userId').size()
print(f"Notes par utilisateur : min={user_stats.min()}, max={user_stats.max()}, médiane={user_stats.median()}")

# 4. Sparsité de la matrice
n_users = ratings['userId'].nunique()
n_movies = ratings['movieId'].nunique()
n_ratings = len(ratings)
sparsity = 1 - (n_ratings / (n_users * n_movies))
print(f"Matrice : {n_users} users x {n_movies} films = {n_users * n_movies:,} cellules")
print(f"Évaluations : {n_ratings:,}")
print(f"Sparsité : {sparsity:.4%} (seulement {1-sparsity:.4%} de cellules remplies !)")

```

CE QU'IL FAUT RETENIR :

- La distribution des notes est asymétrique (plus de notes élevées)
- Certains films ont des milliers d'évaluations, d'autres très peu
- La matrice est EXTREMEMENT SPARSE (>98% de cases vides) → on utilisera des matrices creuses

 **Temps estimé :** 40 minutes

Étape 1.5 — Notebook de Préparation (CRÉER la matrice)

QUOI : Créer la matrice utilisateur-film avec `pivot_table`

QUI : notebooks/04_preparation.ipynb

COMMENT :

```

# Création de la matrice utilisateur-film
user_movie_matrix = ratings.pivot_table(
    index='userId',
    columns='movieId',
    values='rating'
).fillna(0)

print(f"Forme de la matrice : {user_movie_matrix.shape}")
print(f"Nombre de valeurs non-nulles : {(user_movie_matrix > 0).sum().sum()}")

# Sauvegarder pour le notebook de modélisation
import joblib
joblib.dump(user_movie_matrix, "../models/user_movie_matrix.pkl")
joblib.dump(movies, "../models/movies_df.pkl")

```

```
print("✅ Matrice sauvegardée !")
```

POURQUOI C'EST IMPORTANT :

- La `pivot_table` transforme les données longues (1 ligne par note) en format large (1 ligne par utilisateur)
- `fillna(0)` remplit les films non notés par 0 (indispensable pour le calcul)
- Cette matrice est la base de TOUT le système de recommandation

 **Temps estimé :** 20 minutes

Étape 1.6 — Notebook de Modélisation (ENTRAÎNER le modèle)

QUOI : Construire et entraîner le modèle de recommandation

QUI : `notebooks/05_modelisation.ipynb` ET `backend/recommender.py`

COMMENT (étape par étape) :

```
# ÉTAPE 1 : Charger les données préparées
import joblib
import pandas as pd
import numpy as np
from scipy.sparse import csr_matrix
from sklearn.metrics.pairwise import cosine_similarity

ratings = pd.read_csv("../data/ml-latest-small/ratings.csv")
movies = pd.read_csv("../data/ml-latest-small/movies.csv")

# ÉTAPE 2 : Créer la matrice pivot
user_movie_matrix = ratings.pivot_table(
    index="userId", columns="movieId", values="rating"
).fillna(0)
print(f"Matrice : {user_movie_matrix.shape}")

# ÉTAPE 3 : Convertir en matrice creuse (gain mémoire !)
sparse_matrix = csr_matrix(user_movie_matrix.values)
print(f"Taille dense : {user_movie_matrix.values.nbytes / 1e6:.1f} MB")
print(f"Taille sparse : {sparse_matrix.data.nbytes / 1e6:.1f} MB")

# ÉTAPE 4 : Calculer la similarité cosinus
similarity_matrix = cosine_similarity(sparse_matrix)
print(f"Matrice de similarité : {similarity_matrix.shape}")

# ÉTAPE 5 : Tester avec un utilisateur
user_id = 1
user_idx = list(user_movie_matrix.index).index(user_id)
user_similarities = similarity_matrix[user_idx]

# Top 5 utilisateurs similaires
top_users = np.argsort(user_similarities)[::-1][1:6]
print(f"\nUtilisateurs les plus similaires à l'utilisateur {user_id} :")
for i, uid in enumerate(top_users):
```

```

    print(f" {i+1}. User {list(user_movie_matrix.index)[uid]} (similarité:
{user_similarities[uid]:.3f})")

# ÉTAPE 6 : Importer et utiliser la classe du projet
import sys
sys.path.insert(0, "../backend")
from recommender import MovieRecommender

# Créer et entraîner le modèle
recommender = MovieRecommender()
recommender.fit(ratings, movies)

# Tester les recommandations
recommendations = recommender.recommend(user_id=1, n_recommendations=5)
print(f"\nRecommandations pour l'utilisateur {user_id} :")
for i, rec in enumerate(recommendations):
    print(f" {i+1}. {rec['title']} (prédit: {rec['predicted_rating']:.1f})")

# ÉTAPE 7 : Sauvegarder le modèle
recommender.save("../models/recommendation_model.pkl")
print(f"\n✅ Modèle sauvegardé dans models/recommendation_model.pkl")

```

POURQUOI CHAQUE ÉTAPE EST IMPORTANTE :

1. **Matrice pivot** : Transforme les données en format exploitable
2. **Matrice creuse** : Passe de Go à Mo en mémoire
3. **Similarité cosinus** : Mesure la proximité entre utilisateurs
4. **Voisins similaires** : Base du filtrage collaboratif
5. **Moyenne pondérée** : Prédit la note d'un film non noté
6. **Sauvegarde** : Le modèle peut être chargé par l'API plus tard

Le coeur du modèle (dans `backend/recommender.py`) :

```

class MovieRecommender:
    def fit(self, ratings_df, movies_df):
        # 1. Créer la matrice utilisateur-film
        self.user_movie_matrix = ratings_df.pivot_table(
            index="userId", columns="movieId", values="rating"
        ).fillna(0)

        # 2. Calculer la similarité cosinus
        sparse_matrix = csr_matrix(self.user_movie_matrix.values)
        self.similarity_matrix = cosine_similarity(sparse_matrix)

    def recommend(self, user_id, n_recommendations=5):
        # 1. Trouver l'index de l'utilisateur
        user_idx = self.users_ids.tolist().index(user_id)
        user_ratings = self.user_movie_matrix.iloc[user_idx].values

        # 2. Films déjà notés
        rated_movie_indices = np.where(user_ratings > 0)[0]

```

```

# 3. Top 10 utilisateurs similaires
similar_users = np.argsort(self.similarity_matrix[user_idx][:,-1][1:11])

# 4. Prédire pour chaque film non noté
predictions = []
for movie_idx in range(len(user_ratings)):
    if movie_idx in rated_movie_indices:
        continue
    similar_ratings = self.user_movie_matrix.iloc[similar_users, movie_idx].values
    valid = similar_ratings > 0
    if valid.sum() >= 2:
        weights = self.similarity_matrix[user_idx][similar_users][valid]
        predicted = np.average(similar_ratings[valid], weights=weights)
        predictions.append({"movieId": ..., "predicted_rating": predicted})

# 5. Trier et retourner les meilleurs
return sorted(predictions, key=lambda x: x["predicted_rating"], reverse=True)
[:n_recommendations]

```

 **Temps estimé :** 1-2 heures

Étape 1.7 — Notebook d'Évaluation (VALIDER le modèle)

QUOI : Mesurer la qualité des prédictions

QUI : notebooks/06_evaluation.ipynb

COMMENT :

```

from sklearn.metrics import mean_squared_error, mean_absolute_error

# 1. Séparer les données d'entraînement et de test
# (20% des évaluations mises de côté pour le test)

# 2. Calculer les métriques
predictions = [...] # Notes prédites
actuals = [...] # Notes réelles

rmse = np.sqrt(mean_squared_error(actuals, predictions))
mae = mean_absolute_error(actuals, predictions)

print(f"RMSE : {rmse:.3f} (plus petit = mieux)")
print(f"MAE : {mae:.3f} (plus petit = mieux)")
print(f"Note moyenne du dataset : {np.mean(actuals):.2f}")
print(f"Erreur relative : {mae / np.mean(actuals) * 100:.1f}%")

```

CE QU'IL FAUT COMPRENDRE :

- RMSE < 1.0 est acceptable, < 0.8 est bon
- MAE < 0.8 est bon (l'erreur moyenne est inférieure à 1 étoile)
- La sparsité rend l'évaluation difficile (peu de données de test)

 **Temps estimé :** 30 minutes

Phase 2 : La Base de Données (Jour 3)

RÔLE : Data Engineer Junior **OBJECTIF :** Configurer PostgreSQL et créer les tables **FICHIERS CONCERNÉS :**
`backend/database.py` , `backend/models.py` , `backend/seed_data.py`

Étape 2.1 — Démarrer PostgreSQL

OPTION A — Avec Docker (recommandé) :

```
# Démarrer uniquement PostgreSQL
docker run --name movie-db \
  -e POSTGRES_DB=movie_recommender \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_PASSWORD=postgres \
  -p 5432:5432 \
  -d postgres:15-alpine
```

OPTION B — Sans Docker :

1. Installer PostgreSQL depuis [postgresql.org](https://www.postgresql.org)
2. Lancer pgAdmin
3. Créer une base de données `movie_recommender`

VÉRIFICATION :

```
# Connexion à PostgreSQL
docker exec -it movie-db psql -U postgres -d movie_recommender

# Dans psql, taper :
\l          # Liste des bases de données
\q          # Quitter
```

 **Temps estimé :** 15 minutes

Étape 2.2 — Comprendre les modèles et les tables

QUOI : Étudier `backend/models.py` pour comprendre la structure

QUI : `backend/models.py`

CE QU'IL FAUT COMPRENDRE :

```
# 4 tables dans la base de données :

# 1. users : Les utilisateurs de l'application
#   - id (clé primaire)
#   - username (unique, indexé)
#   - email (unique, indexé)
#   - password_hash (hash bcrypt du mot de passe)
#   - created_at (date d'inscription)
```

```
# 2. movies : Les films du catalogue
#   - id (clé primaire = movieId de MovieLens)
#   - title
#   - genres

# 3. ratings : Les évaluations
#   - id (clé primaire)
#   - user_id → FOREIGN KEY vers users(id)
#   - movie_id → FOREIGN KEY vers movies(id)
#   - rating (entre 0.5 et 5.0)
#   - created_at

# 4. recommendations : Les recommandations générées (cachée)
#   Utilisée pour le cache des recommandations
```

 **Temps estimé :** 15 minutes

Étape 2.3 — Créer les tables et importer les données

QUOI : Exécuter le script d'importation

COMMENT :

```
# Méthode 1 : Avec Docker (backend + db doivent tourner)
docker-compose up -d db
docker-compose run --rm backend python seed_data.py --movies /app/data/ml-latest-small/movies.csv --ratings /app/data/ml-latest-small/ratings.csv

# Méthode 2 : Directement avec Python
cd backend
pip install -r requirements.txt
python seed_data.py --movies ../data/ml-latest-small/movies.csv --ratings ../data/ml-latest-small/ratings.csv
```

VÉRIFICATION :

```
# Connexion à PostgreSQL
docker exec -it movie-db psql -U postgres -d movie_recommender

# Dans psql :
\dt                # Doit afficher 4 tables
SELECT COUNT(*) FROM movies;      # ~9 000 films
SELECT COUNT(*) FROM ratings;     # ~100 000 évaluations
SELECT COUNT(*) FROM users;       # ~700 utilisateurs
\q
```

 **Temps estimé :** 15 minutes

Phase 3 : L'API Backend (Jours 3-5)

RÔLE : Backend Developer Junior **OBJECTIF :** Comprendre et lancer l'API FastAPI **FICHIERS CONCERNÉS :**

`backend/main.py` , `backend/crud.py` , `backend/schemas.py` , `backend/auth.py` ,
`backend/database.py`

Étape 3.1 — Étudier l'architecture du backend

QUOI : Lire et comprendre chaque fichier du backend

ORDRE DE LECTURE RECOMMANDÉ :

1. `backend/database.py` (5 lignes) : Configuration de connexion PostgreSQL

```
# Comprendre : comment on se connecte à la base de données
DATABASE_URL = os.getenv("DATABASE_URL",
"postgresql://postgres:postgres@localhost:5432/movie_recommender")
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()
```

2. `backend/models.py` (50 lignes) : Les tables de la base de données

```
# Comprendre : la structure des 4 tables
class Movie(Base): ...
class User(Base): ...
class Rating(Base): ...
class Recommendation(Base): ...
```

3. `backend/schemas.py` (50 lignes) : Les schémas de validation API

```
# Comprendre : comment les données sont validées
class RatingCreate(BaseModel): ...
class UserCreate(BaseModel): ...
```

4. `backend/crud.py` (120 lignes) : Les opérations sur la base de données

```
# Comprendre : les fonctions qui interagissent avec la BDD
def get_movies(db, skip, limit): ...
def create_rating(db, rating): ...
```

5. `backend/auth.py` (80 lignes) : L'authentification JWT

```
# Comprendre : comment les tokens sont créés et vérifiés
def create_access_token(data): ...
def verify_token(token): ...
```

6. `backend/main.py` (150 lignes) : Les routes de l'API

```
# Comprendre : les endpoints disponibles
@app.get("/movies")          # GET /movies?skip=0&limit=20
```

```
@app.get("/movies/{id}")      # GET /movies/42
@app.get("/movies/search")    # GET /movies/search?q=inception
@app.post("/ratings")         # POST /ratings (body: {user_id, movie_id, rating})
@app.get("/recommendations")  # GET /recommendations
@app.post("/users")           # POST /users (body: {username, email, password})
@app.post("/login")           # POST /login (body: {email, password})
@app.get("/stats")            # GET /stats
@app.get("/health")           # GET /health
```

🕒 **Temps estimé : 1 heure**

Étape 3.2 — Lancer le backend et tester

COMMENT :

```
# Option 1 : Avec Docker (backend + db ensemble)
docker-compose up backend

# Option 2 : Directement (si PostgreSQL tourne déjà)
cd backend
pip install -r requirements.txt
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

VÉRIFICATION :

```
# Tester que l'API répond
curl http://localhost:8000/
# → {"message": "API de recommandation de films", "version": "1.0.0", "status": "online"}

curl http://localhost:8000/movies?limit=3
# → [{"id": 1, "title": "Toy Story
(1995)", "genres": "Adventure|Animation|Children|Comedy|Fantasy"}, ...]

curl http://localhost:8000/health
# → {"status": "healthy", "model_loaded": false, "database": "connected"}
```

🔥 **TEST ULTIME :** Ouvrir <http://localhost:8000/docs> dans le navigateur → Vous devriez voir l'interface Swagger UI avec TOUS les endpoints documentés → Vous pouvez tester chaque endpoint directement depuis le navigateur !

🕒 **Temps estimé : 30 minutes**

Phase 4 : Le Frontend (Jours 5-7)

RÔLE : Frontend Developer Junior **OBJECTIF :** Comprendre et lancer l'interface utilisateur **FICHIERS CONCERNÉS :** frontend/

Étape 4.1 — Étudier l'architecture du frontend

QUOI : Lire et comprendre chaque fichier du frontend

ORDRE DE LECTURE RECOMMANDÉ :

1. `frontend/services/api.ts` (100 lignes) : Le client HTTP

```
// Comprendre : comment le frontend communique avec le backend
export const api = {
  login: (email, password) => ...,
  getMovies: (skip, limit) => ...,
  rateMovie: (userId, movieId, rating) => ...,
  getRecommendations: (userId, n) => ...,
}
```

2. `frontend/components/Navbar.tsx` : Barre de navigation
3. `frontend/components/MovieCard.tsx` : Carte de film (60 lignes)
4. `frontend/components/MovieSearch.tsx` : Recherche (80 lignes)
5. `frontend/components/RatingStars.tsx` : Étoiles de notation (50 lignes)
6. `frontend/components/RecommendationsList.tsx` : Liste de recommandations (70 lignes)
7. `frontend/app/page.tsx` : Page d'accueil (150 lignes)
8. `frontend/app/layout.tsx` : Layout racine
9. `frontend/app/login/page.tsx` : Page de connexion
10. `frontend/app/register/page.tsx` : Page d'inscription
11. `frontend/app/profile/page.tsx` : Page de profil
12. `frontend/app/movies/[id]/page.tsx` : Page détail film

🕒 **Temps estimé** : 1 heure

Étape 4.2 — Lancer le frontend

COMMENT :

```
# Option 1 : Avec Docker
# S'assurer que le backend tourne déjà
docker-compose up frontend

# Option 2 : Directement
cd frontend
npm install
npm run dev
```

VÉRIFICATION :

1. Ouvrir <http://localhost:3000>
2. Vous devriez voir la page d'accueil avec le catalogue
3. Cliquer sur un film → page de détail
4. Cliquer sur "Connexion" → page de connexion
5. Créer un compte → noter un film → voir les recommandations

🕒 **Temps estimé :** 15 minutes

Phase 5 : Mise en Production (Jours 7-8)

RÔLE : DevOps Junior **OBJECTIF :** Faire tourner tous les services ensemble **FICHIERS CONCERNÉS :** `docker-compose.yml`, `backend/Dockerfile`, `frontend/Dockerfile`

Étape 5.1 — Démarrer tous les services avec Docker Compose

COMMENT :

```
# 1. Build et démarrage de TOUS les services
docker-compose up --build

# 2. Ou en arrière-plan
docker-compose up --build -d

# 3. Voir les logs
docker-compose logs -f

# 4. Voir l'état des services
docker-compose ps
```

VÉRIFICATION :

```
# Backend
docker-compose ps
# Doit afficher 3 services : db (Up), backend (Up), frontend (Up)

# Tester
curl http://localhost:8000/health
curl http://localhost:3000 # Dans le navigateur
```

🕒 **Temps estimé :** 10 minutes (premier build : 5-10 minutes)

Étape 5.2 — Charger le modèle ML dans l'API

COMMENT :

Si `models/recommendation_model.pkl` existe après la Phase 1 :

```
# Vérifier que le modèle est bien chargé
curl http://localhost:8000/health
# → model_loaded: true
```

Si le modèle n'est pas chargé :

1. Exécuter le notebook `05_modelisation.ipynb`
2. Ou copier manuellement le fichier dans `models/`

🕒 **Temps estimé :** 5 minutes

Étape 5.3 — Tester le parcours complet

PARCOURS UTILISATEUR COMPLET :

1. Ouvrir <http://localhost:3000>
2. Voir le catalogue de films (~9 000 films)
3. Utiliser la barre de recherche (ex: "Inception")
4. Cliquer sur "Connexion" → "S'inscrire"
5. Créer un compte (username, email, password)
6. Naviguer dans le catalogue
7. Cliquer sur un film → page de détail
8. Noter le film (1-5 étoiles)
9. Noter au moins 3 films
10. Aller sur "Mon profil"
11. Voir les recommandations personnalisées !

🕒 **Temps estimé** : 15 minutes

Phase 6 : Git et Déploiement (Jour 8)

RÔLE : DevOps Junior **OBJECTIF** : Sauvegarder et partager le projet

Étape 6.1 — Configurer Git

```
# Initialiser Git (si pas déjà fait)
git init

# Configurer le fichier .gitignore (ne JAMAIS commiter les secrets)
echo ".env" >> .gitignore
echo "__pycache__/" >> .gitignore
echo "*.pyc" >> .gitignore
echo "node_modules/" >> .gitignore
echo "data/" >> .gitignore
echo ".venv/" >> .gitignore
echo "venv/" >> .gitignore
```

Étape 6.2 — Premier commit

```
git add -A
git commit -m "feat: initialisation du système de recommandation de films"

# Push vers GitHub (créer d'abord un repo sur github.com)
git remote add origin https://github.com/votre-compte/movie-recommender.git
git push -u origin main
```

Étape 6.3 — Commandes Git quotidiennes

```
# Avant de commencer à travailler
git pull
```

```
# Après avoir fait des changements
git status          # Voir ce qui a changé
git diff            # Voir les modifications en détail
git add nom_du_fichier.py  # Stager un fichier
git commit -m "description"  # Commiter

# Envoyer les changements
git push
```

 **Temps estimé : 30 minutes**

Récapitulatif visuel du parcours complet

JOUR 1	JOUR 2-3	JOUR 4-5	JOUR 6-7	JOUR 8
▼	▼	▼	▼	▼
ENVIRONNEMENT	DATA SCIENCE	BACKEND API	FRONTEND	GIT & FIN
• Python • Node.js • Docker • PostgreSQL • Git	• Import • Nettoyage • Analyse • Modèle ML • Évaluation	• FastAPI • SQLAlchemy • JWT Auth • CRUD • Routes API	• Next.js • Composants • Axios • Tailwind • UI/UX	• Commit • Push • Fin 🎉

Checklist de réussite

Cochez chaque case au fur et à mesure :

Phase 0 : Prérequis

- ☐ Python 3.10+ installé (`python --version`)
- ☐ Node.js 20+ installé (`node --version`)
- ☐ Docker Desktop installé (`docker --version`)
- ☐ Git installé (`git --version`)
- ☐ VS Code installé
- ☐ Projet cloné et exploré
- ☐ Fichier `.env` créé

Phase 1 : Data Science

- ☐ Dataset MovieLens téléchargé et extrait dans `data/`
- ☐ Importation des données (notebook 01)
- ☐ Nettoyage des données (notebook 02)
- ☐ Analyse exploratoire (notebook 03)

- ☐ Préparation de la matrice (notebook 04)
- ☐ Modèle entraîné et sauvegardé (notebook 05 + `recommender.py`)
- ☐ Évaluation du modèle (notebook 06)

Phase 2 : ☒ Base de Données

- ☐ PostgreSQL démarré (Docker ou local)
- ☐ Tables créées (models.py)
- ☐ Données importées (seed_data.py)
- ☐ Vérification : ~9 000 films, ~100 000 évaluations

Phase 3 : ☒ Backend

- ☐ Schemas et modèles compris
- ☐ API FastAPI lancée (`curl http://localhost:8000/health`)
- ☐ Swagger UI accessible (`http://localhost:8000/docs`)
- ☐ Endpoints testés (movies, ratings, users)

Phase 4 : ☒ Frontend

- ☐ Frontend lancé (`http://localhost:3000`)
- ☐ Catalogue visible
- ☐ Inscription et connexion fonctionnelles
- ☐ Notation de films possible

Phase 5 : ☒ Intégration

- ☐ Docker Compose fait tout tourner
- ☐ Modèle ML chargé (`model_loaded: true`)
- ☐ Parcours complet fonctionnel
- ☐ Recommandations personnalisées visibles

Phase 6 : ☒ Finalisation

- ☐ Projet commité dans Git
- ☐ Projet pushé sur GitHub

Dépannage rapide (FAQ)

Problème	Cause probable	Solution
connection refused sur localhost:8000	Backend pas lancé	<code>cd backend && uvicorn main:app --reload</code>
relation "movies" does not exist	Base de données vide	<code>python seed_data.py</code>
No module named 'jose'	Dépendance manquante	<code>pip install python-jose[cryptography]</code>
Port 5432 already in use	PostgreSQL déjà lancé	<code>docker stop movie-db</code> ou changer de port
Model not loaded	Fichier .pkl manquant	Lancer le notebook 05_modelisation

401 Unauthorized	Token manquant ou expiré	Se reconnecter
Network Error dans le frontend	API non accessible	Vérifier NEXT_PUBLIC_API_URL

3. Python pour la Data Science

2.1 Pourquoi Python ?

Python est le **langage roi de la Data Science** pour plusieurs raisons :

1. **Syntaxe simple et lisible** — Idéal pour les débutants
2. **Écosystème riche** — Des milliers de bibliothèques spécialisées
3. **Communauté immense** — Documentation, tutoriels, forums
4. **Polyvalence** — Du notebook Jupyter à la production

2.2 Concepts Python essentiels pour ce projet

2.2.1 Les types de base

```
# Types numériques
entier = 42          # int
decimal = 3.14       # float
nombre_complexe = 1+2j # complex

# Types texte
texte = "Hello World" # str

# Types booléens
vrai = True          # bool
faux = False

# Types collections
liste = [1, 2, 3]     # list (modifiable)
tuple_ = (1, 2, 3)    # tuple (immutable)
dictionnaire = {"cle": "valeur"} # dict
ensemble = {1, 2, 3}  # set (valeurs uniques)
```

2.2.2 Les structures de contrôle

```
# Conditionnelle
if note >= 4:
    print("Excellent film !")
elif note >= 3:
    print("Bon film")
else:
    print("Film moyen")
```

```

# Boucle for
for i in range(5):      # 0, 1, 2, 3, 4
    print(i)

for film in films:
    print(film["titre"])

# Boucle while
compteur = 0
while compteur < 10:
    compteur += 1

# List comprehension (ultra-utile en Data Science)
carres = [x**2 for x in range(10)]      # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
films_recents = [f for f in films if f.annee > 2020]

```

2.2.3 Les fonctions

```

# Fonction simple
def moyenne(liste):
    return sum(liste) / len(liste)

# Fonction avec type hints (recommandé !)
def calculer_note_moyenne(notes: list[float]) -> float:
    """Calcule la moyenne d'une liste de notes.

    Args:
        notes: Liste des notes (entre 0 et 5)

    Returns:
        La moyenne arrondie à 2 décimales
    """
    return round(sum(notes) / len(notes), 2)

# Fonction lambda (fonction anonyme, une ligne)
carre = lambda x: x ** 2
films_tries = sorted(films, key=lambda f: f.note, reverse=True)

```

2.2.4 Les classes (POO)

```

class Film:
    """Représente un film dans notre système."""

    def __init__(self, titre: str, annee: int, genre: str):
        self.titre = titre
        self.annee = annee
        self.genre = genre
        self.notes = []

```

```

def ajouter_note(self, note: float):
    """Ajoute une note au film."""
    if 0 <= note <= 5:
        self.notes.append(note)

def note_moyenne(self) -> float:
    """Calcule la note moyenne du film."""
    if not self.notes:
        return 0.0
    return sum(self.notes) / len(self.notes)

def __str__(self) -> str:
    return f"{self.titre} ({self.annee}) - {self.genre}"

```

2.3 Modules et imports

```

# Import standard
import math
import os
import json

# Import avec alias (convention Data Science)
import numpy as np          # Convention universelle
import pandas as pd         # Convention universelle
import matplotlib.pyplot as plt # Convention universelle

# Import de modules spécifiques
from sklearn.metrics.pairwise import cosine_similarity
from sqlalchemy.orm import Session
from typing import List, Optional

```

2.4 Gestion des erreurs (try/except)

```

try:
    resultat = 10 / 0
except ZeroDivisionError:
    print("Division par zéro !")
except Exception as e:
    print(f"Erreur inattendue : {e}")
finally:
    print("Ce bloc s'exécute toujours")

```

2.5 Travailler avec les fichiers

```

# Lecture d'un fichier CSV (la manière classique)
with open("data/films.csv", "r", encoding="utf-8") as f:
    contenu = f.read()

```

```
# Écriture
with open("resultats.txt", "w") as f:
    f.write("Résultat de l'analyse")

# JSON
with open("data/config.json", "r") as f:
    config = json.load(f)
```

4. NumPy

3.1 Qu'est-ce que NumPy ?

NumPy (Numerical Python) est la bibliothèque fondamentale pour le calcul scientifique en Python. Elle introduit :

- **Les ndarray** : Des tableaux multidimensionnels performants
- **Les opérations vectorisées** : Des calculs sans boucles Python
- **L'algèbre linéaire** : Matrices, dot products, décompositions
- **Les fonctions statistiques** : Moyenne, écart-type, percentiles

💡 **Pourquoi c'est important ?** *Pandas est construit SUR NumPy. Scikit-learn utilise NumPy. Comprendre NumPy = comprendre la base de TOUTE la Data Science Python.*

3.2 Créer des tableaux (ndarray)

```
import numpy as np

# Depuis une liste Python
arr = np.array([1, 2, 3, 4, 5])      # 1D array
matrice = np.array([[1, 2], [3, 4]]) # 2D array

# Tableaux prédéfinis
zeros = np.zeros((3, 4))            # Matrice 3x4 de zéros
ones = np.ones((2, 3))              # Matrice 2x3 de 1
identite = np.eye(5)                # Matrice identité 5x5
aleatoire = np.random.rand(3, 3)    # Nombres aléatoires uniformes [0, 1]
normal = np.random.randn(1000)      # Distribution normale centrée réduite

# Séquences
sequence = np.arange(0, 10, 2)       # [0, 2, 4, 6, 8]
lignes = np.linspace(0, 1, 5)        # [0.0, 0.25, 0.5, 0.75, 1.0]
```

3.3 Les attributs importants d'un array

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

arr.shape      # (2, 3) → dimensions du tableau
```

```

arr.ndim      # 2 → nombre de dimensions
arr.size      # 6 → nombre total d'éléments
arr.dtype     # int64 → type de données
arr.T         # Transposée → array([[1,4],[2,5],[3,6]])
arr.flat      # Itérateur sur tous les éléments

```

3.4 Indexation et slicing

```

arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Indexation classique
arr[0]          # [1, 2, 3] (première ligne)
arr[0, 1]       # 2 (ligne 0, colonne 1)
arr[-1]         # [7, 8, 9] (dernière ligne)

# Slicing
arr[0:2]        # [[1,2,3],[4,5,6]] (lignes 0 et 1)
arr[:, 0:2]     # [[1,2],[4,5],[7,8]] (colonnes 0 et 1)
arr[:, ::2]     # [[1,3],[7,9]] (un pas sur 2)

# Indexation booléenne (TRÈS UTILE)
masque = arr > 5
arr[masque]     # [6, 7, 8, 9] (tous les éléments > 5)

# Indexation par liste d'indices
indices = [0, 2]
arr[indices]    # [[1,2,3],[7,8,9]]

```

3.5 Le Broadcasting (concept clé)

Le **broadcasting** est ce qui rend NumPy si puissant. Il permet d'effectuer des opérations entre des tableaux de formes différentes.

```

# Règle : Les dimensions sont compatibles si elles sont égales ou si l'une vaut 1

# Exemple 1 : Scalaires
arr = np.array([1, 2, 3])
arr + 10      # [11, 12, 13] → 10 est "broadcasté" sur tout le tableau

# Exemple 2 : 1D + 2D
a = np.array([[1, 2, 3], [4, 5, 6]]) # shape (2, 3)
b = np.array([10, 20, 30])          # shape (3,) → broadcasté en (1, 3) → (2, 3)
a + b                                # [[11,22,33],[14,25,36]]

# Exemple 3 : Normalisation (cas réel Data Science)
notes = np.array([[3.5, 4.0, 2.5],
                  [5.0, 1.0, 3.0],
                  [2.0, 3.0, 4.0]])

```

```
moyennes_colonnes = notes.mean(axis=0) # Moyenne par colonne
notes_centrees = notes - moyennes_colonnes # Broadcasting automatique !
```

3.6 Opérations vectorisées (CRUCIAL pour la performance)

```
# ❌ MAUVAISE pratique (boucle Python lente)
resultat = []
for i in range(len(grand_tableau)):
    resultat.append(grand_tableau[i] * 2 + 1)

# ✅ BONNE pratique (vectorisée, 10-100x plus rapide)
resultat = grand_tableau * 2 + 1

# Opérations arithmétiques
arr + 10      # Addition
arr - 5       # Soustraction
arr * 2       # Multiplication
arr / 3       # Division
arr ** 2      # Puissance
np.sqrt(arr)  # Racine carrée
np.exp(arr)   # Exponentielle
np.log(arr)   # Logarithme
np.sin(arr)   # Sinus

# Opérations de comparaison
arr > 3        # Tableau booléen
(arr > 2) & (arr < 5) # ET logique
(arr < 2) | (arr > 4) # OU logique
```

3.7 Algèbre linéaire

```
# Produit matriciel (DOT PRODUCT)
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

np.dot(a, b)      # Produit matriciel classique
a @ b             # Syntaxe Python 3.5+ (identique à np.dot)

# Produit élément par élément
a * b             # [[5,12],[21,32]]

# Transposée
a.T

# Normes (utile pour la similarité cosinus !)
v = np.array([3, 4])
np.linalg.norm(v) # 5.0 (norme euclidienne =  $\sqrt{3^2+4^2}$ )
```

```
# Décomposition SVD (Singular Value Decomposition)
U, S, Vt = np.linalg.svd(matrice)

# Inversion de matrice
np.linalg.inv(matrice)
```

3.8 Fonctions statistiques

```
donnees = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

np.mean(donnees)          # 5.5 (moyenne)
np.median(donnees)        # 5.5 (médiane)
np.std(donnees)           # 2.87 (écart-type)
np.var(donnees)           # 8.25 (variance)
np.min(donnees)           # 1
np.max(donnees)           # 10
np.percentile(donnees, 25) # 3.25 (1er quartile)
np.percentile(donnees, 75) # 7.75 (3ème quartile)

# Avec l'axe (axis) – TRÈS IMPORTANT
matrice = np.array([[1, 2, 3], [4, 5, 6]])
np.mean(matrice, axis=0)  # [2.5, 3.5, 4.5] → moyenne PAR COLONNE
np.mean(matrice, axis=1)  # [2.0, 5.0] → moyenne PAR LIGNE
```

3.9 Où trouver les fonctions (référence rapide)

```
# Création
np.array()      # Depuis une liste
np.zeros()      # Rempli de zéros
np.ones()       # Rempli de 1
np.eye()        # Matrice identité
np.arange()     # Séquence régulière
np.linspace()   # Intervalle linéaire
np.random.rand() # Aléatoire uniforme
np.random.randn() # Aléatoire normal

# Manipulation
.reshape()      # Changer la forme
.ravel()        # Aplatir en 1D
.T             # Transposée
np.concatenate() # Concaténer
np.stack()      # Empiler

# Mathématiques
np.sum()        # Somme
np.mean()       # Moyenne
np.std()        # Écart-type
np.dot()        # Produit scalaire
```

```

np.linalg.inv() # Inverse
np.linalg.norm() # Norme
np.linalg.svd() # SVD

# Tri et recherche
np.sort()      # Trier
np.argsort()   # Indices du tri
np.where()     # Indices des conditions
np.unique()    # Valeurs uniques

```

3.10 Dans notre projet (cas concrets)

```

# Dans recommender.py :
# 1. Trouver les indices des films déjà notés
rated_movie_indices = np.where(user_ratings > 0)[0]

# 2. Trier les utilisateurs par similarité
similar_users = np.argsort(user_similarities)[::-1][1:11]

# 3. Moyenne pondérée des notes
predicted = np.average(ratings, weights=weights)

```

5. Pandas

4.1 Qu'est-ce que Pandas ?

Pandas est la bibliothèque #1 pour la manipulation et l'analyse de données tabulaires en Python. Elle introduit deux structures de données fondamentales :

- **Series** : Une colonne étiquetée (1D)
- **DataFrame** : Un tableau étiqueté (2D), similaire à une feuille Excel ou une table SQL

💡 **Pandas est construit sur NumPy.** Chaque colonne d'un **DataFrame** est une série **NumPy**.

4.2 Les deux structures de données

```

import pandas as pd
import numpy as np

### Series (1D)
notes = pd.Series([3.5, 4.0, 2.5, 5.0],
                  index=["Film1", "Film2", "Film3", "Film4"])

# Film1    3.5
# Film2    4.0
# Film3    2.5
# Film4    5.0

```

```

### DataFrame (2D) – LA structure principale
films = pd.DataFrame({
    "titre": ["Inception", "Matrix", "Interstellar"],
    "annee": [2010, 1999, 2014],
    "note": [4.8, 4.5, 4.7],
    "genre": ["Sci-Fi", "Action", "Sci-Fi"]
})
#      titre  annee  note  genre
# 0  Inception   2010   4.8  Sci-Fi
# 1    Matrix   1999   4.5  Action
# 2 Interstellar  2014   4.7  Sci-Fi

```

4.3 Modules essentiels — Guide complet

4.3.1 Importation de données

```

# CSV (le plus courant en Data Science)
df = pd.read_csv("data/films.csv")
df = pd.read_csv("data/films.csv", sep=";", encoding="utf-8")
df = pd.read_csv("data/ratings.csv",
                 usecols=["userId", "movieId", "rating"], # Colonnes à charger
                 nrows=10000) # Limiter le nombre de lignes

# Excel
df = pd.read_excel("data/films.xlsx", sheet_name="Films")

# JSON
df = pd.read_json("data/films.json")

# SQL (très utile pour ce projet)
from sqlalchemy import create_engine
engine = create_engine("postgresql://user:pass@localhost/db")
df = pd.read_sql("SELECT * FROM movies", engine)
df = pd.read_sql_query("SELECT * FROM movies WHERE year > 2000", engine)
df = pd.read_sql_table("movies", engine)

# Depuis une liste de dictionnaires
donnees = [
    {"titre": "Inception", "annee": 2010},
    {"titre": "Matrix", "annee": 1999},
]
df = pd.DataFrame(donnees)

```

4.3.2 Exploration des données

```

# Premier aperçu
df.head()           # 5 premières lignes
df.head(10)         # 10 premières lignes
df.tail()           # 5 dernières lignes

```

```

df.sample(5)          # 5 lignes aléatoires

# Informations générales
df.info()             # Types, valeurs non-nulles, mémoire
df.describe()         # Statistiques descriptives (count, mean, std, min, max, quartiles)
df.describe(include="all") # Pour toutes les colonnes (incluant catégorielles)
df.shape              # (n_lignes, n_colonnes)
df.columns            # Liste des colonnes
df.dtypes             # Types des colonnes
df.index              # Index du DataFrame

# Valeurs uniques et comptages
df["genre"].unique()   # Valeurs uniques d'une colonne
df["genre"].value_counts() # Comptage de chaque valeur
df["genre"].nunique()  # Nombre de valeurs uniques

```

4.3.3 Sélection et filtrage

```

# Sélection de colonnes
df["titre"]           # Une colonne → Series
df[["titre", "note"]] # Plusieurs colonnes → DataFrame
df.titre              # Accès attribut (attention aux espaces)

# Sélection de lignes par position (.iloc)
df.iloc[0]            # Première ligne
df.iloc[1:5]          # Lignes 1 à 4
df.iloc[[0, 2, 4]]    # Lignes 0, 2, 4
df.iloc[1:3, 0:2]     # Lignes 1-2, colonnes 0-1

# Sélection de lignes par étiquette (.loc)
df.loc[0]             # Ligne d'index 0
df.loc[0:5]           # Lignes 0 à 5 (INCLUSIF !)
df.loc[df["note"] > 4.5] # Toutes les lignes avec note > 4.5

# Filtrage booléen (LE PLUS UTILE)
df[df["note"] > 4.0]   # Films notés > 4
df[(df["note"] > 4.0) & (df["annee"] > 2000)] # ET logique
df[(df["genre"] == "Sci-Fi") | (df["genre"] == "Action")] # OU logique
df[df["titre"].str.contains("Inception")]    # Recherche texte

# Méthode .query() (alternative élégante)
df.query("note > 4.0 and annee > 2000")
df.query("genre in ['Sci-Fi', 'Action']")

```

4.3.4 Nettoyage des données

```

# Valeurs manquantes
df.isna()           # Masque booléen des valeurs manquantes
df.isna().sum()     # Comptage par colonne

```

```

df.dropna()                # Supprimer les lignes avec des NaN
df.dropna(subset=["note"])  # Supprimer si "note" est NaN
df.fillna(0)               # Remplacer NaN par 0
df.fillna(df.mean())       # Remplacer par la moyenne
df["note"] = df["note"].fillna(df["note"].median()) # Remplacer par la médiane

# Doublons
df.duplicated()            # Détecter les doublons
df.drop_duplicates()       # Supprimer les doublons
df.drop_duplicates(subset=["titre", "annee"]) # Basé sur certaines colonnes

# Conversion de types
df["annee"] = df["annee"].astype(int)
df["date"] = pd.to_datetime(df["date"])
df["note"] = pd.to_numeric(df["note"], errors="coerce") # "coerce" → NaN si erreur

# Renommer
df.rename(columns={"titre": "title", "note": "rating"}, inplace=True)

# Réindexer
df.reset_index(drop=True)  # Réinitialiser l'index
df.set_index("movieId")    # Définir une colonne comme index

```

4.3.5 GroupBy — L'OPÉRATION LA PLUS PUISSANTE

Le **GroupBy** suit le principe "Split-Apply-Combine" :

1. **Split** : Diviser les données en groupes
2. **Apply** : Appliquer une fonction à chaque groupe
3. **Combine** : Combiner les résultats

```

# Exemple : Note moyenne par genre
df.groupby("genre")["note"].mean()

# Plusieurs colonnes
df.groupby("genre").agg({
    "note": ["mean", "std", "count"],
    "annee": "min"
})

# Plusieurs agrégations nommées
df.groupby("genre").agg(
    note_moyenne=("note", "mean"),
    nb_films=("titre", "count"),
    note_max=("note", "max")
)

# GroupBy multiple (ex: par genre ET année)
df.groupby(["genre", "annee"])["note"].mean()

# Transformation (apply)

```

```
df.groupby("genre")["note"].transform("mean") # Ajoute la moyenne du genre à chaque ligne

# Filtrer les groupes
df.groupby("genre").filter(lambda x: len(x) > 10) # Garde les genres avec > 10 films
```

4.3.6 Fusion de DataFrames (comme les JOINS SQL)

```
# merge() = JOIN SQL
# left: films, right: ratings
films_notes = pd.merge(
    left=films,
    right=ratings,
    left_on="movieId",
    right_on="movieId",
    how="inner" # inner, left, right, outer
)

# Types de jointures
pd.merge(df1, df2, on="cle") # INNER JOIN (défaut)
pd.merge(df1, df2, on="cle", how="left") # LEFT JOIN
pd.merge(df1, df2, on="cle", how="right") # RIGHT JOIN
pd.merge(df1, df2, on="cle", how="outer") # FULL OUTER JOIN

# Concaténation (empiler)
pd.concat([df1, df2]) # Empiler verticalement (rows)
pd.concat([df1, df2], axis=1) # Empiler horizontalement (columns)
```

4.3.7 Pivot Tables (Tableaux croisés dynamiques)

Fonction CRUCIALE pour notre projet ! La `pivot_table` crée la matrice utilisateur-film.

```
# Matrice utilisateur-film (chaque ligne = utilisateur, chaque colonne = film)
user_movie_matrix = ratings.pivot_table(
    index="userId", # Lignes = utilisateurs
    columns="movieId", # Colonnes = films
    values="rating", # Valeurs = notes
    fill_value=0 # Remplacer NaN par 0
)

# Moyenne des notes par utilisateur et genre
note_par_genre = ratings.merge(films, on="movieId").pivot_table(
    index="userId",
    columns="genre",
    values="rating",
    aggfunc="mean" # Par défaut: "mean"
)

# melt() – Inverse de pivot_table (passe de large à long)
long_df = pd.melt(
    frame=user_movie_matrix.reset_index(),
```

```

    id_vars="userId",
    var_name="movieId",
    value_name="rating"
)

```

4.3.8 Apply et Map (Transformations)

```

# apply() – Applique une fonction à chaque ligne/colonne
df["note_arrondie"] = df["note"].apply(round)
df["categorie"] = df["note"].apply(
    lambda x: "Excellent" if x >= 4 else "Bon" if x >= 3 else "Moyen"
)

# apply sur plusieurs colonnes
df[["note_min", "note_max"]] = df[["note_1", "note_2", "note_3"]].apply(
    lambda row: pd.Series([row.min(), row.max()]), axis=1
)

# map() – Substitution de valeurs
df["genre_code"] = df["genre"].map({
    "Action": 1,
    "Comedy": 2,
    "Drama": 3,
    "Sci-Fi": 4
})

# replace() – Alternative à map pour les remplacements
df["genre"] = df["genre"].replace("Sci-Fi", "Science Fiction")

```

4.3.9 Manipulation avancée

```

# Trier
df.sort_values("note", ascending=False)          # Par note décroissante
df.sort_values(["genre", "note"], ascending=[True, False])

# Ajouter / Supprimer des colonnes
df["nouvelle_colonne"] = ...                      # Nouvelle colonne
df["is_recent"] = df["annee"] >= 2020            # Colonne booléenne
df.drop("colonne_a_supprimer", axis=1)           # Supprimer une colonne

# Colonnes calculées
df["decennie"] = (df["annee"] // 10) * 10
df["note_ponderee"] = df["note"] * df["nb_votes"]

# Travailler avec les dates
df["date"] = pd.to_datetime(df["date"])
df["annee"] = df["date"].dt.year
df["mois"] = df["date"].dt.month
df["jour_semaine"] = df["date"].dt.day_name()

```

```
# Travailler avec le texte
df["titre_minuscule"] = df["titre"].str.lower()
df["premier_mot"] = df["titre"].str.split().str[0]
df["contient_inception"] = df["titre"].str.contains("inception", case=False)
```

4.3.10 Exportation

```
df.to_csv("resultats.csv", index=False)
df.to_csv("resultats.csv", sep=";", encoding="utf-8-sig") # Pour Excel
df.to_excel("resultats.xlsx", sheet_name="Films")
df.to_json("resultats.json", orient="records")
df.to_sql("table_name", engine, if_exists="replace", index=False)
df.to_parquet("resultats.parquet") # Format efficace pour gros volumes
df.to_pickle("resultats.pkl") # Format pickle
```

4.4 Dans notre projet (cas concrets)

```
# Dans seed_data.py :
# Lecture du CSV MovieLens
df = pd.read_csv(ratings_path)

# Dans recommender.py :
# Création de la matrice pivot utilisateur-film
self.user_movie_matrix = ratings_df.pivot_table(
    index="userId", columns="movieId", values="rating"
).fillna(0)

# Dans crud.py :
# Bulk import avec pandas
df = pd.read_csv(csv_path)
```

4.5 Pièges courants à éviter

```
# ⚠ Attention au chaining (modification en chaîne)
df[df["note"] > 4]["new_col"] = 1 # ❌ NE MARCHE PAS (SettingWithCopyWarning)
df.loc[df["note"] > 4, "new_col"] = 1 # ✅ BONNE PRATIQUE

# ⚠ inplace=True (préférer l'assignation)
df.drop("col", axis=1, inplace=True) # ❌ Déconseillé
df = df.drop("col", axis=1) # ✅ Recommandé

# ⚠ Boucles sur DataFrame (lent !)
for i in range(len(df)): # ❌ TRÈS LENT
    df.iloc[i, "new"] = df.iloc[i, "col"] * 2
df["new"] = df["col"] * 2 # ✅ 100x plus rapide
```

6. SciPy

5.1 Qu'est-ce que SciPy ?

SciPy (Scientific Python) est la bibliothèque pour les mathématiques avancées et l'algorithmique scientifique. Elle étend NumPy avec des modules spécialisés.

💡 **Dans notre projet**, SciPy est crucial pour les **matrices creuses** (`scipy.sparse`), essentielles dans les systèmes de recommandation où la matrice utilisateur-film est très sparse (99%+ de zéros).

5.2 Les matrices creuses (scipy.sparse)

5.2.1 Pourquoi les matrices creuses ?

Dans un système de recommandation :

- 10 000 utilisateurs × 100 000 films = **1 milliard de cellules**
- Mais un utilisateur note en moyenne 50 films = **0.005% de cellules remplies**
- Une matrice dense prendrait **8 Go** en mémoire
- Une matrice creuse prend **quelques Mo**

5.2.2 Les formats disponibles

```
from scipy.sparse import csr_matrix, csc_matrix, coo_matrix

# CSR (Compressed Sparse Row) – RECOMMANDÉ pour notre cas
# Optimisé pour : multiplication matricielle, lecture de lignes, slicing par lignes
sparse_csr = csr_matrix(matrice_dense)

# CSC (Compressed Sparse Column) – Pour slicing par colonnes
sparse_csc = csc_matrix(matrice_dense)

# COO (Coordinate) – Bon pour la construction
# Format : (row, col, value)
lignes = [0, 0, 1, 1, 2]
colonnes = [0, 2, 1, 3, 0]
valeurs = [5, 3, 4, 2, 1]
sparse_coo = coo_matrix((valeurs, (lignes, colonnes)), shape=(3, 4))

# Conversion entre formats
sparse_csr = sparse_coo.tocsr()
sparse_csc = sparse_csr.tocsc()
matrice_dense = sparse_csr.toarray() # ⚠ NE FAITES ÇA QUE SI LA MÉMOIRE LE PERMET
```

5.2.3 Opérations sur matrices creuses

```
# Attributs importants
sparse_csr.shape      # (n_lignes, n_colonnes)
sparse_csr.nnz        # Nombre de valeurs non-nulles
sparse_csr.data       # Les valeurs non-nulles
```

```

sparse_csr.indices      # Les indices de colonnes
sparse_csr.indptr       # Les pointeurs de lignes

# Opérations (identique à NumPy !)
sparse_csr * 2          # Multiplication scalaire
sparse_csr + sparse_csr # Addition
sparse_csr @ sparse_vector # Produit matrice-vecteur
sparse_csr.T            # Transposée

# Similarité cosinus (manière efficace)
from sklearn.metrics.pairwise import cosine_similarity
similarity = cosine_similarity(sparse_csr) # Accepte les matrices creuses !

```

5.3 Algèbre linéaire avancée (scipy.linalg vs np.linalg)

```

from scipy import linalg

# SVD tronquée sur matrice creuse
from scipy.sparse.linalg import svds
U, S, Vt = svds(sparse_matrix, k=50) # k = nombre de facteurs latents

# Résolution de systèmes linéaires
x = linalg.solve(A, b)

# Décomposition QR
Q, R = linalg.qr(A)

# Valeurs propres
eigenvalues, eigenvectors = linalg.eig(A)

```

5.4 Statistiques (scipy.stats)

```

from scipy import stats

# Distributions de probabilité
notes = np.array([3.5, 4.0, 2.5, 5.0, 3.0])
stats.describe(notes) # Statistiques descriptives

# Tests statistiques
stats.ttest_ind(groupe1, groupe2) # Test t de Student
stats.pearsonr(x, y)               # Corrélation de Pearson
stats.spearmanr(x, y)              # Corrélation de Spearman

# Distribution normale
stats.norm.cdf(1.96)               # Fonction de répartition
stats.norm.ppf(0.975)              # Quantile inverse
stats.norm.rvs(size=1000)          # Échantillon aléatoire

```

5.5 Dans notre projet (cas concrets)

```
# Dans recommender.py :
# Conversion de la matrice dense pandas en format sparse pour le calcul
sparse_matrix = csr_matrix(self.user_movie_matrix.values)

# Calcul de similarité cosinus (efficace car sparse)
self.similarity_matrix = cosine_similarity(sparse_matrix)
```

7. Scikit-learn

6.1 Qu'est-ce que Scikit-learn ?

Scikit-learn est la bibliothèque de Machine Learning la plus populaire pour Python. Elle offre :

- Des algorithmes ML prêts à l'emploi
- Des outils de prétraitement des données
- Des métriques d'évaluation
- Une API cohérente (tous les modèles suivent le même pattern)

6.2 L'API unifiée (Le pattern fit/predict)

Le concept le plus important à comprendre : Tous les modèles scikit-learn suivent le même pattern :

```
# 1. Créer le modèle
model = NomDuModele(hyperparamètres)

# 2. Entraîner le modèle
model.fit(X_train, y_train)

# 3. Faire des prédictions
predictions = model.predict(X_test)

# 4. Évaluer
score = model.score(X_test, y_test)
```

6.3 Les modules essentiels

6.3.1 train_test_split — Division des données

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,           # 20% pour le test, 80% pour l'entraînement
    random_state=42,         # Pour la reproductibilité
    stratify=y               # Préserve la proportion des classes
```

```
)

# Pourquoi random_state ? Sans lui, à chaque exécution :
# - La division est différente
# - Les résultats ne sont pas reproductibles
# - Le débogage est impossible
```

6.3.2 Preprocessing — Préparation des données

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder

# StandardScaler : moyenne=0, écart-type=1 (pour les modèles linéaires, SVM, etc.)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)  #  $Z = (X - \mu) / \sigma$ 

# MinMaxScaler : valeurs entre 0 et 1 (pour les réseaux de neurones)
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)  #  $Z = (X - \min) / (\max - \min)$ 

# LabelEncoder : transformer du texte en nombres
encoder = LabelEncoder()
y_encoded = encoder.fit_transform(["Sci-Fi", "Action", "Sci-Fi", "Drama"])
# → [2, 0, 2, 1]
```

6.3.3 Metrics — Évaluation des modèles

```
from sklearn.metrics import (
    mean_squared_error,      # Erreur quadratique moyenne
    mean_absolute_error,     # Erreur absolue moyenne
    r2_score,                # Coefficient de détermination  $R^2$ 
    accuracy_score,          # Précision (classification)
    confusion_matrix         # Matrice de confusion
)

# Régression (prédire des valeurs numériques)
mse = mean_squared_error(y_true, y_pred)  # Plus petit = mieux
rmse = np.sqrt(mse)                      # En unités originales
mae = mean_absolute_error(y_true, y_pred)  # Plus robuste que MSE
r2 = r2_score(y_true, y_pred)             # 1 = parfait, 0 = moyen, négatif = mauvais

# Classification (prédire des catégories)
accuracy = accuracy_score(y_true, y_pred)  # Proportion de bonnes prédictions
```

6.3.4 cosine_similarity — LE CŒUR DE NOTRE PROJET

```
from sklearn.metrics.pairwise import cosine_similarity

# Similarité cosinus = mesure l'angle entre deux vecteurs
```

```
# cos(θ) = (A · B) / (||A|| × ||B||)
# Valeur : -1 (opposé) à 1 (identique), 0 = indépendant

# Exemple avec notre projet :
# Vecteurs utilisateurs (leurs notes sur tous les films)
user_1_ratings = np.array([5, 0, 3, 0, 4]) # A noté film 1, 3, 5
user_2_ratings = np.array([4, 0, 4, 0, 5]) # Goûts similaires
user_3_ratings = np.array([1, 0, 5, 0, 1]) # Goûts différents

# Calcul de similarité
similarity_matrix = cosine_similarity([user_1_ratings, user_2_ratings, user_3_ratings])
# [[1.0, 0.99, 0.78],    User 1 est très similaire à User 2
#  [0.99, 1.0, 0.74],    User 2 est similaire à User 1
#  [0.78, 0.74, 1.0 ]]   User 3 est moins similaire

# Pourquoi cosinus plutôt qu'euclidien ?
# Cosinus ignore l'échelle de notation :
# User A : [1, 2] (note sévèrement)
# User B : [5, 10] (note généreusement)
# Distance euclidienne = grande (car échelle différente)
# Cosinus ≈ 1 (car les PROFILS sont identiques)
```

6.3.5 Pipeline — Organisation du workflow

```
from sklearn.pipeline import Pipeline

# Une pipeline enchaîne transformations + modèle
pipeline = Pipeline([
    ("scaler", StandardScaler()),    # Étape 1 : normalisation
    ("model", KNeighborsRegressor()), # Étape 2 : modèle
])

pipeline.fit(X_train, y_train)
predictions = pipeline.predict(X_test)

# Avantages :
# 1. Code plus propre
# 2. Pas de fuite de données (data leakage)
# 3. Facile à déployer
```

6.4 Joblib — Sauvegarde et chargement des modèles

```
import joblib

# Sauvegarder un modèle
joblib.dump(model, "models/recommendation_model.pkl")
joblib.dump({
    "model": model,
    "scaler": scaler,
```

```

    "features": feature_names
}, "models/complete_pipeline.pkl")

# Charger un modèle
model = joblib.load("models/recommendation_model.pkl")

# Pourquoi joblib et pas pickle ?
# - Meilleure performance sur les gros tableaux NumPy
# - Compression automatique possible
# - Plus robuste

```

6.5 Dans notre projet (cas concrets)

```

# Dans recommender.py :
# LE module utilisé est cosine_similarity
from sklearn.metrics.pairwise import cosine_similarity

# Calcul de la matrice de similarité entre utilisateurs
self.similarity_matrix = cosine_similarity(sparse_matrix)

# joblib pour la persistance
import joblib

# Sauvegarde
joblib.dump(self, filepath)

# Chargement
recommender = joblib.load(filepath)

```

8. PostgreSQL

7.1 Qu'est-ce que PostgreSQL ?

PostgreSQL (souvent appelé "Postgres") est un système de gestion de base de données relationnelle open-source, reconnu pour :

- Sa **fiabilité** (utilisé en production depuis 30+ ans)
- Sa **conformité aux standards SQL**
- Ses **performances** et son optimisation des requêtes
- Ses **extensions** puissantes

7.2 Concepts fondamentaux

7.2.1 Base de données et tables

```

DATABASE : movie_recommender

```

users	movies	ratings
id	id	id
username	title	user_id
email	genres	movie_id
password		rating
created_at		created

7.2.2 Types de données PostgreSQL

```
-- Numériques
INTEGER          -- Entiers (comme movieId)
SERIAL           -- Auto-incrémenté (comme PRIMARY KEY)
FLOAT / REAL     -- Nombres décimaux (comme rating)
NUMERIC(3,1)     -- Décimal précis (type DECIMAL)

-- Texte
VARCHAR(255)     -- Chaîne de longueur limitée (comme title)
TEXT            -- Texte long (comme description)
CHAR(1)          -- Caractère fixe

-- Temps
DATE            -- Date seulement
TIME           -- Heure seulement
TIMESTAMP      -- Date + heure (comme created_at)
TIMESTAMPPTZ   -- Date + heure avec fuseau

-- Autres
BOOLEAN        -- Vrai/Faux
JSON / JSONB   -- Données JSON (JSONB est indexable)
UUID           -- Identifiant universel
ARRAY          -- Tableaux
```

7.2.3 Contraintes SQL

```
-- PRIMARY KEY : Identifiant unique d'une table
id INTEGER PRIMARY KEY

-- FOREIGN KEY : Lien vers une autre table
user_id INTEGER REFERENCES users(id) ON DELETE CASCADE

-- UNIQUE : Valeur unique dans la colonne
email VARCHAR(100) UNIQUE

-- NOT NULL : Champ obligatoire
title VARCHAR(255) NOT NULL
```

```
-- CHECK : Validation personnalisée
CONSTRAINT check_rating_range CHECK (rating >= 0.5 AND rating <= 5.0)

-- DEFAULT : Valeur par défaut
created_at TIMESTAMP DEFAULT NOW()
```

7.3 Commandes SQL essentielles

7.3.1 Création et modification

```
-- Créer une base de données
CREATE DATABASE movie_recommender;

-- Créer une table
CREATE TABLE movies (
    id INTEGER PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    genres VARCHAR(255) NOT NULL
);

-- Modifier une table
ALTER TABLE movies ADD COLUMN release_year INTEGER;
ALTER TABLE movies DROP COLUMN release_year;
ALTER TABLE movies ALTER COLUMN title TYPE TEXT;
```

7.3.2 Lecture (SELECT)

```
-- Sélection de base
SELECT * FROM movies;
SELECT title, genres FROM movies;

-- Filtrage
SELECT * FROM movies WHERE genres LIKE '%Sci-Fi%';
SELECT * FROM movies WHERE id = 42;
SELECT * FROM movies WHERE title ILIKE '%inception%';

-- Agrégation
SELECT genres, COUNT(*) as nb_films, AVG(r.rating) as note_moyenne
FROM movies m
JOIN ratings r ON m.id = r.movie_id
GROUP BY genres
HAVING COUNT(*) > 10
ORDER BY note_moyenne DESC;

-- Pagination (utilisé dans notre API)
SELECT * FROM movies ORDER BY id LIMIT 20 OFFSET 0;
SELECT * FROM movies ORDER BY id LIMIT 20 OFFSET 20;

-- Jointures
```

```
SELECT u.username, m.title, r.rating
FROM ratings r
JOIN users u ON r.user_id = u.id
JOIN movies m ON r.movie_id = m.id
WHERE u.id = 1
ORDER BY r.rating DESC;
```

7.3.3 Insertion

```
-- Insertion simple
INSERT INTO movies (id, title, genres) VALUES (1, 'Inception', 'Sci-Fi|Action');

-- Insertion multiple
INSERT INTO movies (id, title, genres) VALUES
(1, 'Inception', 'Sci-Fi|Action'),
(2, 'The Matrix', 'Action|Sci-Fi'),
(3, 'Interstellar', 'Sci-Fi|Drama');
```

7.3.4 Mise à jour et suppression

```
-- Mise à jour
UPDATE ratings SET rating = 4.5 WHERE user_id = 1 AND movie_id = 42;

-- Suppression
DELETE FROM ratings WHERE user_id = 1;
DELETE FROM movies; -- Attention ! Supprime tout

-- Tronquer (supprime tout, plus rapide que DELETE)
TRUNCATE TABLE ratings;
```

7.4 Gestion des utilisateurs et permissions

```
-- Créer un utilisateur
CREATE USER mon_app WITH PASSWORD 'mot_de_passe_secure';

-- Donner les permissions
GRANT CONNECT ON DATABASE movie_recommender TO mon_app;
GRANT USAGE ON SCHEMA public TO mon_app;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO mon_app;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO mon_app;

-- Voir les bases de données
\l

-- Voir les tables
\dt
```

```
-- Voir la structure d'une table
\d movies
```

7.5 Connexion avec Python

```
import psycopg2
from sqlalchemy import create_engine
import pandas as pd

# Avec psycopg2 (bas niveau)
conn = psycopg2.connect(
    host="localhost",
    port=5432,
    database="movie_recommender",
    user="postgres",
    password="postgres"
)
cur = conn.cursor()
cur.execute("SELECT * FROM movies LIMIT 5")
rows = cur.fetchall()
cur.close()
conn.close()

# Avec SQLAlchemy (recommandé, utilisé dans le projet)
DATABASE_URL = "postgresql://postgres:postgres@localhost:5432/movie_recommender"
engine = create_engine(DATABASE_URL)

# Lire directement dans un DataFrame !
df = pd.read_sql("SELECT * FROM movies", engine)

# Écrire un DataFrame dans la base de données
df.to_sql("movies_temp", engine, if_exists="replace", index=False)
```

7.6 pgAdmin — L'interface graphique

pgAdmin est installé avec PostgreSQL et permet de :

1. **Visualiser** la structure de la base de données
2. **Exécuter** des requêtes SQL
3. **Gérer** les utilisateurs et permissions
4. **Importer/Exporter** des données
5. **Surveiller** les performances

Pour y accéder : <http://localhost:5050> ou via le menu Démarrer → pgAdmin 4

7.7 Bonnes pratiques

```
-- Indexer les colonnes fréquemment utilisées dans WHERE
CREATE INDEX idx_ratings_user_id ON ratings(user_id);
```

```

CREATE INDEX idx_ratings_movie_id ON ratings(movie_id);
CREATE INDEX idx_movies_title ON movies USING gin(to_tsvector('french', title));

-- Utiliser EXPLAIN pour analyser les requêtes lentes
EXPLAIN ANALYZE SELECT * FROM ratings WHERE user_id = 42;

-- Transaction : garantir l'intégrité des données
BEGIN;
UPDATE users SET ... ;
INSERT INTO ratings ... ;
COMMIT; -- ou ROLLBACK en cas d'erreur

-- Backup régulier
pg_dump -U postgres movie_recommender > backup.sql

```

9. SQLAlchemy

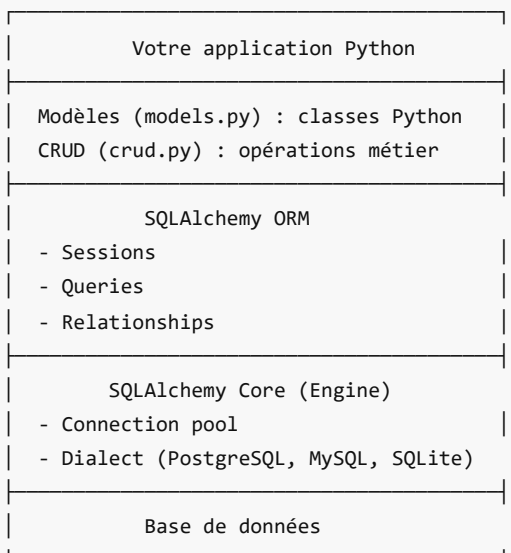
8.1 Qu'est-ce que SQLAlchemy ?

SQLAlchemy est l'ORM (Object-Relational Mapping) le plus utilisé en Python. Il permet de :

- **Mapper** des classes Python en tables de base de données
- **Interagir** avec la BDD sans écrire de SQL brut
- **Changer** de base de données sans changer le code

💡 *Dans notre projet, SQLAlchemy connecte l'API FastAPI à PostgreSQL.*

8.2 Architecture de SQLAlchemy



8.3 Configuration de base

```

from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

# 1. L'URL de connexion
DATABASE_URL = "postgresql://postgres:postgres@localhost:5432/movie_recommender"

# 2. Le moteur (gère la connexion)
engine = create_engine(DATABASE_URL)

# 3. La factory de sessions
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# 4. La classe de base pour les modèles
Base = declarative_base()

```

8.4 Définir des modèles (models.py)

```

from sqlalchemy import Column, Integer, String, Float, DateTime, ForeignKey
from sqlalchemy.orm import relationship
from sqlalchemy.sql import func
from .database import Base

class Movie(Base):
    __tablename__ = "movies" # Nom de la table en BDD

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String(255), nullable=False)
    genres = Column(String(255), nullable=False)

    # Relations
    ratings = relationship("Rating", back_populates="movie")

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    username = Column(String(50), unique=True, nullable=False, index=True)
    email = Column(String(100), unique=True, nullable=False, index=True)
    password_hash = Column(String(255), nullable=False)
    created_at = Column(DateTime(timezone=True), server_default=func.now())

    ratings = relationship("Rating", back_populates="user")

class Rating(Base):
    __tablename__ = "ratings"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"), nullable=False)

```

```

movie_id = Column(Integer, ForeignKey("movies.id", ondelete="CASCADE"), nullable=False)
rating = Column(Float, nullable=False)
created_at = Column(DateTime(timezone=True), server_default=func.now())

user = relationship("User", back_populates="ratings")
movie = relationship("Movie", back_populates="ratings")

```

8.5 Les types de colonnes SQLAlchemy

Column(Integer)	# INTEGER
Column(String(255))	# VARCHAR(255)
Column(Text)	# TEXT
Column(Float)	# FLOAT/REAL
Column(Boolean)	# BOOLEAN
Column(DateTime)	# TIMESTAMP WITHOUT TIME ZONE
Column(DateTime(timezone=True))	# TIMESTAMP WITH TIME ZONE
Column(Date)	# DATE
Column(Enum("A", "B", "C"))	# ENUM
Column(JSON)	# JSON
Column.ARRAY(Integer)	# INTEGER[]

8.6 Contraintes et arguments

```

Column(Integer, primary_key=True)      # Clé primaire
Column(String, unique=True)            # UNIQUE
Column(String, nullable=False)         # NOT NULL
Column(String, index=True)             # INDEX
Column(Integer, ForeignKey("users.id")) # FOREIGN KEY
Column(String, default="default_value") # DEFAULT
Column(DateTime, server_default=func.now()) # DEFAULT côté serveur

# Check constraint
__table_args__ = (
    CheckConstraint("rating >= 0.5 AND rating <= 5.0"),
    UniqueConstraint("user_id", "movie_id"),
)

```

8.7 Les relations entre tables

```

# One-to-Many (Un utilisateur → Plusieurs évaluations)
class User(Base):
    ratings = relationship("Rating", back_populates="user")

class Rating(Base):
    user_id = Column(ForeignKey("users.id"))
    user = relationship("User", back_populates="ratings")

```

```
# Paramètres de relationship :
# - back_populates : Lien bidirectionnel
# - lazy="select" : Chargé à la demande (défaut)
# - lazy="joined" : Toujours chargé (JOIN)
# - cascade="all, delete-orphan" : Supprime les enfants si le parent est supprimé
```

8.8 Les sessions (le cœur des opérations)

```
from .database import SessionLocal

# Créer une session
db = SessionLocal()

try:
    # Opérations...
    db.add(objet)
    db.commit()
    db.refresh(objet) # Recharger depuis la BDD (obtient l'ID auto-généré)
except:
    db.rollback() # Annuler en cas d'erreur
finally:
    db.close() # Toujours fermer !
```

Le pattern avec FastAPI (Dependency Injection)

```
from fastapi import Depends
from sqlalchemy.orm import Session
from .database import SessionLocal

def get_db():
    db = SessionLocal()
    try:
        yield db # FastAPI fournit db à la route
    finally:
        db.close() # Et ferme automatiquement après

@app.get("/movies")
def get_movies(db: Session = Depends(get_db)):
    return db.query(Movie).all()
```

8.9 Opérations CRUD complètes

CREATE

```
# Création
new_movie = Movie(id=42, title="Inception", genres="Sci-Fi|Action")
db.add(new_movie)
db.commit()
```

```

db.refresh(new_movie) # Maintenant new_movie.id est rempli

# Création en masse (bulk)
movies = [
    Movie(id=1, title="Inception", genres="Sci-Fi|Action"),
    Movie(id=2, title="The Matrix", genres="Action|Sci-Fi"),
]
db.add_all(movies)
db.commit()

# Méthode utilitaire dans crud.py
def create_movie(db: Session, movie: schemas.MovieCreate) -> models.Movie:
    db_movie = models.Movie(**movie.model_dump())
    db.add(db_movie)
    db.commit()
    db.refresh(db_movie)
    return db_movie

```

READ

```

# Tous les films
movies = db.query(Movie).all()

# Premier résultat
movie = db.query(Movie).first()

# Filtrer
movie = db.query(Movie).filter(Movie.id == 42).first()
movies = db.query(Movie).filter(Movie.genres.like("%Sci-Fi%")).all()

# Recherche (ILKE = insensible à la casse)
movies = db.query(Movie).filter(Movie.title.ilike(f"%{query}%")).all()

# Ordonner
movies = db.query(Movie).order_by(Movie.title).all()
movies = db.query(Movie).order_by(Movie.title.desc()).all()

# Pagination
movies = db.query(Movie).offset(skip).limit(limit).all()

# Compter
count = db.query(Movie).count()
count = db.query(Movie).filter(Movie.genres.like("%Sci-Fi%")).count()

```

UPDATE

```

# 1. Récupérer l'objet
movie = db.query(Movie).filter(Movie.id == 42).first()

```

```
# 2. Modifier ses attributs
movie.title = "Inception (2010)"
movie.genres = "Sci-Fi|Action|Thriller"

# 3. Sauvegarder
db.commit()
db.refresh(movie)
```

DELETE

```
# Supprimer un objet
movie = db.query(Movie).filter(Movie.id == 42).first()
if movie:
    db.delete(movie)
    db.commit()

# Supprimer directement
db.query(Rating).filter(Rating.user_id == 1).delete()
db.commit()
```

8.10 Agrégations et fonctions SQL

```
from sqlalchemy import func

# Compter
n_users = db.query(func.count(User.id)).scalar() # Retourne le nombre
n_movies = db.query(User).count()                # Alternative

# Moyenne
avg_rating = db.query(func.avg(Rating.rating)).scalar()

# Grouper
results = db.query(
    Movie.genres,
    func.count(Rating.id).label("n_ratings"),
    func.avg(Rating.rating).label("avg_rating")
).join(Rating).group_by(Movie.genres).all()

# Pour chaque résultat :
for row in results:
    print(row.genres, row.n_ratings, row.avg_rating)
```

8.11 Import en masse avec Pandas

```
import pandas as pd

def bulk_import_movies(db: Session, csv_path: str) -> int:
```

```

"""Importe les films depuis un CSV dans la BDD."""
df = pd.read_csv(csv_path)
count = 0

for _, row in df.iterrows():
    # Vérifier si le film existe déjà
    existing = db.query(Movie).filter(Movie.id == row["movieId"]).first()
    if not existing:
        movie = Movie(
            id=row["movieId"],
            title=row["title"],
            genres=row["genres"],
        )
        db.add(movie)
        count += 1

db.commit()
return count

```

8.12 Bonnes pratiques

```

# 1. Toujours utiliser le pattern with ou try/finally
with SessionLocal() as db:
    result = db.query(Movie).all()

# 2. N+1 query problem : charger les relations en une requête
# ❌ MAUVAIS (N+1 requêtes)
ratings = db.query(Rating).all()
for rating in ratings:
    print(rating.user.name) # Requête supplémentaire à chaque itération !

# ✅ BON (2 requêtes ou 1 avec joinedload)
from sqlalchemy.orm import joinedload
ratings = db.query(Rating).options(joinedload(Rating.user)).all()

# 3. Indexer les colonnes de recherche fréquentes
# Dans models.py : Column(String, index=True)

# 4. Utiliser des transactions pour les opérations groupées
try:
    db.add(movie)
    db.add(rating)
    db.commit() # Les deux sont sauvegardés ou aucun
except:
    db.rollback()
    raise

```

10. FastAPI

9.1 Qu'est-ce que FastAPI ?

FastAPI est un framework web moderne et performant pour construire des API avec Python. Ses points forts :

- **Très performant** (comparable à Node.js et Go)
- **Validation automatique** via Pydantic
- **Documentation interactive** générée automatiquement
- **Basé sur les type hints Python** (typage fort)
- **Support natif de l'async**

9.2 Structure d'une application FastAPI

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import Session
from typing import List
import models, schemas, crud
from database import engine, SessionLocal

# Création de l'application
app = FastAPI(
    title="Movie Recommender API",
    description="API de recommandation de films",
    version="1.0.0"
)

# Middleware CORS (essentiel pour le frontend)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],          # En dev, "*" est OK. En prod : ["http://localhost:3000"]
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

9.3 Les routes

9.3.1 Les méthodes HTTP

```
@app.get("/movies")           # Lire
@app.post("/ratings")          # Créer
@app.put("/movies/{id}")       # Mettre à jour (remplacer)
@app.patch("/movies/{id}")     # Mettre à jour (partiellement)
@app.delete("/movies/{id}")    # Supprimer
```

9.3.2 Les paramètres de route

```

# Paramètre de chemin (path parameter)
@app.get("/movies/{movie_id}")
def get_movie(movie_id: int):
    return crud.get_movie(db, movie_id)

# Paramètre de requête (query parameter)
@app.get("/movies")
def get_movies(
    skip: int = 0,          # Valeur par défaut
    limit: int = 20         # Valeur par défaut
):
    return crud.get_movies(db, skip=skip, limit=limit)

# Avec validation
from fastapi import Query

@app.get("/movies/search")
def search_movies(
    q: str = Query(..., min_length=1, max_length=100),
    limit: int = Query(10, ge=1, le=50)
):
    return crud.search_movies(db, query=q, limit=limit)

```

9.3.3 Corps de requête (request body)

```

from pydantic import BaseModel

class RatingCreate(BaseModel):
    user_id: int
    movie_id: int
    rating: float = Field(..., ge=0.5, le=5.0)

@app.post("/ratings")
def create_rating(rating: RatingCreate, db: Session = Depends(get_db)):
    # FastAPI valide automatiquement le corps de la requête
    # Si rating est invalide, une erreur claire est renvoyée
    return crud.create_rating(db, rating)

```

9.3.4 Réponses et codes HTTP

```

from fastapi import HTTPException, status

@app.get("/movies/{movie_id}")
def get_movie(movie_id: int, db: Session = Depends(get_db)):
    movie = crud.get_movie(db, movie_id)
    if not movie:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Film avec ID {movie_id} non trouvé"
        )

```

```

    )
    return movie

# Codes HTTP courants :
# 200 OK          → Succès
# 201 Created     → Ressource créée
# 400 Bad Request → Mauvaise requête (validation)
# 401 Unauthorized → Non authentifié
# 403 Forbidden   → Pas les droits
# 404 Not Found   → Ressource inexistante
# 422 Unprocessable → Erreur de validation Pydantic
# 500 Server Error → Erreur serveur

```

9.4 Le système de dépendances (Dependency Injection)

C'est la fonctionnalité la plus puissante de FastAPI.

```

# Définir une dépendance
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# Utiliser la dépendance dans une route
@app.get("/movies")
def get_movies(db: Session = Depends(get_db)): # ← Injection automatique !
    return db.query(Movie).all()

# Dépendance qui dépend d'une autre dépendance
def get_current_user(
    db: Session = Depends(get_db),
    token: str = Depends(oauth2_scheme)
):
    user = verify_token(token)
    return user

# Utilisation
@app.get("/profile")
def get_profile(current_user: User = Depends(get_current_user)):
    return current_user

```

9.5 Les modèles de réponse

```

from typing import List
from pydantic import BaseModel

# Modèle de réponse (ce qui est renvoyé)

```

```

class MovieResponse(BaseModel):
    id: int
    title: str
    genres: str

    class Config:
        from_attributes = True # Permet de convertir un modèle SQLAlchemy

# Utilisation dans la route
@app.get("/movies", response_model=List[MovieResponse])
def get_movies(skip: int = 0, limit: int = 20):
    movies = crud.get_movies(db, skip=skip, limit=limit)
    return movies # Auto-conversion SQLAlchemy → Pydantic

```

9.6 Middleware CORS

```

from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:3000", # Frontend en dev
        "https://mon-site.com", # Frontend en prod
    ],
    allow_credentials=True,
    allow_methods=["*"], # GET, POST, PUT, DELETE, etc.
    allow_headers=["*"], # Authorization, Content-Type, etc.
)

# ⚠️ "allow_origins=[*]" est pratique en dev mais DANGEREUX en prod

```

9.7 Documentation interactive

FastAPI génère automatiquement une documentation interactive :

- **Swagger UI** : <http://localhost:8000/docs>
- **ReDoc** : <http://localhost:8000/redoc>

Ces interfaces permettent de :

1. Voir toutes les routes disponibles
2. Lire les descriptions et paramètres
3. **Tester les routes directement dans le navigateur !**
4. Voir les schémas de données

9.8 Gestion des fichiers statiques

```

from fastapi.staticfiles import StaticFiles

```

```
# Servir des fichiers statiques (images, CSS, etc.)
app.mount("/static", StaticFiles(directory="static"), name="static")
```

9.9 Routage avancé

```
from fastapi import APIRouter

# Créer un routeur
router = APIRouter(
    prefix="/movies",      # Toutes les routes commencent par /movies
    tags=["movies"]        # Tag dans la documentation
)

@router.get("/")
def list_movies():
    pass

@router.get("/{movie_id}")
def get_movie(movie_id: int):
    pass

# Inclure le routeur dans l'app
app.include_router(router)
```

9.10 Configuration et variables d'environnement

```
import os
from dotenv import load_dotenv

load_dotenv() # Charge .env

DATABASE_URL = os.getenv("DATABASE_URL",
    "postgresql://postgres:postgres@localhost:5432/movie_recommender")
SECRET_KEY = os.getenv("JWT_SECRET_KEY", "change-me-in-production")
```

9.11 Lancer l'application

```
# En développement (avec rechargement automatique)
uvicorn main:app --reload

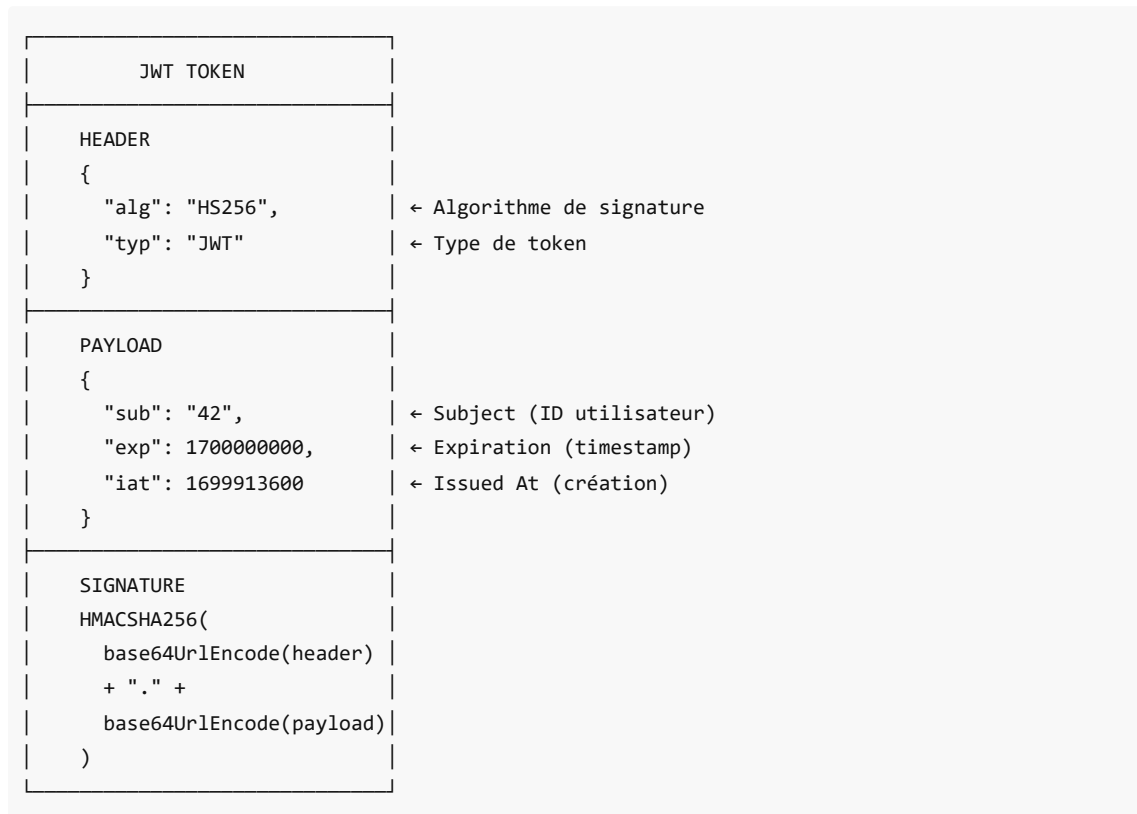
# En production
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4

# Avec Docker (dans le Dockerfile)
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

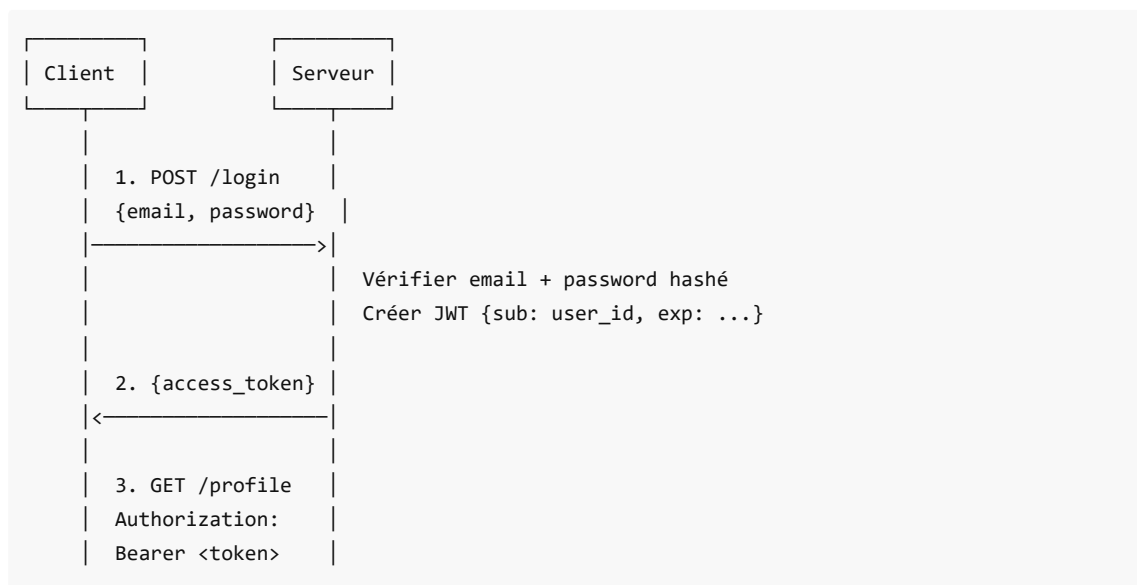
11. JWT & Authentication

10.1 Qu'est-ce que JWT ?

JWT (JSON Web Token) est un standard ouvert (RFC 7519) qui définit un moyen compact et sécurisé de transmettre des informations entre parties sous forme d'un objet JSON.



10.2 Comment ça marche ?





10.3 Implémentation dans notre projet

auth.py — Création et vérification des tokens

```
from datetime import datetime, timedelta, timezone
from jose import JWTError, jwt
from passlib.hash import bcrypt
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer
import os

# Configuration
SECRET_KEY = os.getenv("SECRET_KEY", "change-me-in-production")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24 # 24 heures

security = HTTPBearer(auto_error=False)

def create_access_token(data: dict) -> str:
    """Crée un JWT token."""
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def verify_token(token: str) -> dict | None:
    """Vérifie et décode un JWT token."""
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return payload
    except JWTError:
        return None
```

Le hash des mots de passe

```
from passlib.hash import bcrypt

# Hachage (à l'inscription)
hashed_password = bcrypt.hash("mon_mot_de_passe")
# → "$2b$12$LJ3m4ys3Lk..."
```

```

# Vérification (à la connexion)
bcrypt.verify("mon_mot_de_passe", hashed_password) # True
bcrypt.verify("mauvais_mdp", hashed_password)      # False

# Jamais de stockage en clair !
# user.password_hash = bcrypt.hash(user.password)

```

Les dépendances FastAPI

```

from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials

def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(security)
):
    """Récupère l'utilisateur depuis le token JWT."""
    if credentials is None:
        return None # Route optionnelle

    payload = verify_token(credentials.credentials)
    if payload is None:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Token invalide ou expiré"
        )

    user_id = payload.get("sub")
    # Récupérer l'utilisateur de la BDD...
    return user

def require_user(current_user = Depends(get_current_user)):
    """Exige un utilisateur authentifié."""
    if current_user is None:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Authentification requise"
        )
    return current_user

# Utilisation dans les routes :
@app.get("/public") # Tout le monde peut accéder
def public():
    pass

@app.get("/profile") # Seulement les utilisateurs connectés
def profile(user: User = Depends(require_user)):
    return user

```

10.4 Sécurité — Bonnes pratiques

```
# 1. SECRET_KEY : JAMAIS en dur dans le code
# ❌ MAUVAIS
SECRET_KEY = "ma-super-cle"

# ✅ BON (via variable d'environnement)
import os
SECRET_KEY = os.getenv("JWT_SECRET_KEY")
if not SECRET_KEY:
    raise RuntimeError("JWT_SECRET_KEY non configurée !")

# Générer une clé sécurisée :
# openssl rand -hex 32
# → "a1b2c3d4e5f6..."

# 2. Toujours mettre une expiration
# ❌ MAUVAIS (token valide à vie)
token = jwt.encode({"sub": user_id}, SECRET_KEY)

# ✅ BON
token = jwt.encode({
    "sub": user_id,
    "exp": datetime.now(timezone.utc) + timedelta(hours=24),
    "iat": datetime.now(timezone.utc) # Optionnel : création
}, SECRET_KEY)

# 3. HTTPS en production (obligatoire pour les tokens)
# Les tokens JWT transitent dans les headers HTTP
# Sans HTTPS, ils peuvent être interceptés

# 4. Valider l'entrée utilisateur
# Vérifier que le password a la longueur minimale
if len(password) < 6:
    raise HTTPException(status_code=400, detail="Mot de passe trop court")
```

10.5 Déroulement complet de l'authentification (dans le projet)

```
# 1. Inscription (POST /users)
@app.post("/users")
def create_user(user: UserCreate, db: Session = Depends(get_db)):
    # Vérifier si l'utilisateur existe déjà
    existing = crud.get_user_by_email(db, user.email)
    if existing:
        raise HTTPException(400, "Email déjà utilisé")

    # Créer l'utilisateur (avec mot de passe hashé)
    return crud.create_user(db, user)
```

```

# 2. Connexion (POST /login)
@app.post("/login")
def login(credentials: LoginRequest, db: Session = Depends(get_db)):
    # Vérifier les identifiants
    user = crud.authenticate_user(db, credentials.email, credentials.password)
    if not user:
        raise HTTPException(401, "Email ou mot de passe incorrect")

    # Générer le token JWT
    token = create_access_token(data={"sub": str(user.id)})

    return {
        "access_token": token,
        "token_type": "bearer",
        "user_id": user.id,
        "username": user.username
    }

# 3. Route protégée (GET /recommendations)
@app.get("/recommendations")
def get_recommendations(
    current_user: User = Depends(require_user), # ← Authentification requise
    db: Session = Depends(get_db)
):
    recommendations = recommender.recommend(current_user.id)
    return recommendations

```

12. Pydantic

11.1 Qu'est-ce que Pydantic ?

Pydantic est la bibliothèque de validation de données la plus populaire en Python. Elle permet de :

- **Définir** la structure des données avec des classes Python
- **Valider** automatiquement les types et contraintes
- **Sérialiser/Désérialiser** (JSON ↔ Python)
- **Générer** la documentation automatiquement

💡 **Pydantic est le cœur de FastAPI** : chaque requête et réponse est validée par Pydantic.

11.2 Définir des modèles Pydantic

```

from pydantic import BaseModel, Field, EmailStr
from typing import Optional, List
from datetime import datetime

class UserCreate(BaseModel):
    username: str = Field(..., min_length=3, max_length=50)
    email: str

```

```

password: str = Field(..., min_length=6)

class MovieResponse(BaseModel):
    id: int
    title: str
    genres: str

    class Config:
        from_attributes = True # Permet la conversion SQLAlchemy → Pydantic

class LoginRequest(BaseModel):
    email: str
    password: str

class TokenResponse(BaseModel):
    access_token: str
    token_type: str = "bearer"
    user_id: int
    username: str

```

11.3 Les types de champs

```

from pydantic import BaseModel, Field
from typing import Optional, List
from datetime import datetime

class MovieDetail(BaseModel):
    id: int
    title: str = Field(..., min_length=1, max_length=500)
    genres: str = Field(..., description="Genres séparés par |")
    poster_url: Optional[str] = None # Champ optionnel
    release_year: Optional[int] = Field(None, ge=1900, le=2030)
    rating: float = Field(5.0, ge=0.0, le=10.0) # Valeur par défaut

    class Config:
        from_attributes = True

```

11.4 Les validateurs personnalisés

```

from pydantic import BaseModel, field_validator

class RatingCreate(BaseModel):
    user_id: int
    movie_id: int
    rating: float = Field(..., ge=0.5, le=5.0)

    @field_validator("rating")
    @classmethod

```

```

def validate_rating(cls, v):
    if v % 0.5 != 0:
        raise ValueError("La note doit être un multiple de 0.5")
    return v

@field_validator("user_id")
@classmethod
def validate_user_id(cls, v):
    if v <= 0:
        raise ValueError("ID utilisateur invalide")
    return v

```

11.5 Conversion SQLAlchemy → Pydantic

```

# La magie de from_attributes=True

# Modèle SQLAlchemy (base de données)
class Movie(Base):
    __tablename__ = "movies"
    id = Column(Integer, primary_key=True)
    title = Column(String(255))
    genres = Column(String(255))

# Schéma Pydantic (API)
class MovieResponse(BaseModel):
    id: int
    title: str
    genres: str

    class Config:
        from_attributes = True # Active la conversion automatique

# Dans la route FastAPI
@app.get("/movies/{id}", response_model=MovieResponse)
def get_movie(id: int, db: Session = Depends(get_db)):
    movie = db.query(Movie).filter(Movie.id == id).first()
    return movie # Conversion automatique !

```

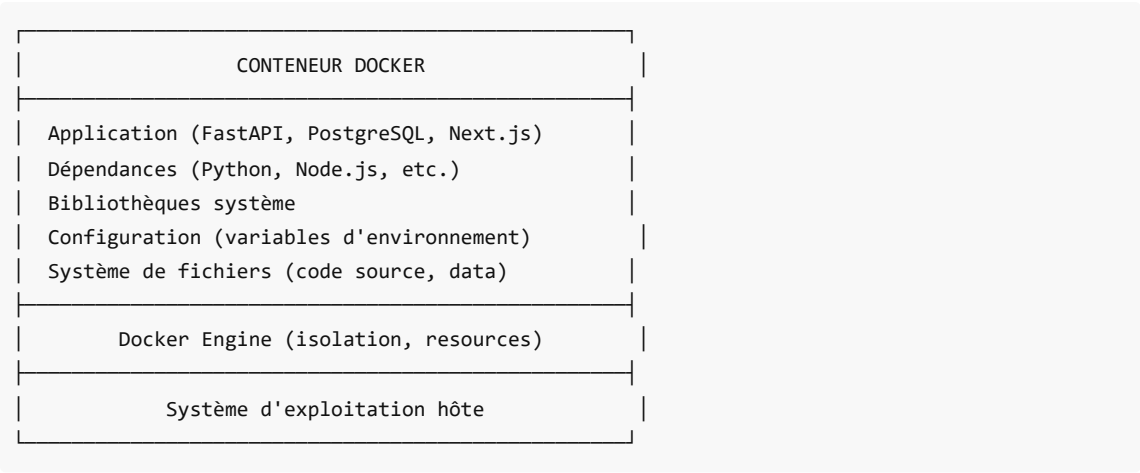
13. Docker

12.1 Qu'est-ce que Docker ?

Docker est une plateforme de conteneurisation qui permet d'empaqueter une application avec toutes ses dépendances dans un **conteneur** standardisé et isolé.

💡 **"Ça marche sur ma machine"** — Avec Docker, ça marche PARTOUT !

12.2 Concepts fondamentaux



Concept	Explication	Analogie
Image	Template en lecture seule pour créer des conteneurs	CD-ROM d'installation
Conteneur	Instance en cours d'exécution d'une image	Une VM allumée
Volume	Stockage persistant partagé entre conteneur et hôte	Disque dur externe
Réseau	Communication entre conteneurs	Câble réseau virtuel
Dockerfile	Recette pour construire une image	Fichier de recette

12.3 Dockerfile — Backend (FastAPI)

```
# 1. Image de base
FROM python:3.11-slim

# 2. Répertoire de travail
WORKDIR /app

# 3. Copier les dépendances d'abord (cache Docker optimisé)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 4. Copier le code source
COPY . .

# 5. Commande de démarrage
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

12.4 Dockerfile — Frontend (Next.js)

```

# Étape 1 : Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Étape 2 : Production (multi-stage)
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/.next ./next
COPY --from=builder /app/public ./public
COPY --from=builder /app/package*.json ./
RUN npm install --production
CMD ["npm", "start"]

```

12.5 Docker Compose — Orchestration

Le fichier `docker-compose.yml` définit et orchestre tous les services :

```

version: "3.8"

services:
  # === Base de données ===
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: movie_recommender
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5

  # === Backend API ===
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://postgres:postgres@db:5432/movie_recommender
      JWT_SECRET_KEY: votre-cle-secrete-ici
    depends_on:

```

```

    db:
      condition: service_healthy # Attend que PostgreSQL soit prêt

# === Frontend ===
frontend:
  build: ./frontend
  ports:
    - "3000:3000"
  environment:
    NEXT_PUBLIC_API_URL: http://localhost:8000
  depends_on:
    - backend

volumes:
  postgres_data: # Volume persistant pour les données

```

12.6 Commandes Docker essentielles

```

# Construction et démarrage de tous les services
docker-compose up --build

# Démarrage en arrière-plan (détaché)
docker-compose up -d

# Arrêt des services
docker-compose down

# Voir les logs
docker-compose logs -f backend
docker-compose logs -f frontend

# Exécuter une commande dans un conteneur
docker-compose exec backend python seed_data.py
docker-compose exec db psql -U postgres -d movie_recommender

# Voir les conteneurs en cours d'exécution
docker ps

# Nettoyer tout (conteneurs, images, volumes)
docker-compose down -v

```

12.7 Astuces Docker pour le développement

```

# Montage de volume pour le hot-reload (développement)
services:
  backend:
    build: ./backend
  volumes:

```

```
- ./backend:/app # Le code local est monté dans le conteneur
command: uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

```
# .dockerignore (NE PAS OUBLIER !)  
.git  
__pycache__  
*.pyc  
.env  
.venv  
node_modules
```

14. Next.js

13.1 Qu'est-ce que Next.js ?

Next.js est un framework React qui offre :

- **Rendu côté serveur (SSR)** : Pages générées sur le serveur pour un meilleur SEO
- **Routage par système de fichiers** : Les pages = des fichiers dans `app/`
- **Server Components** : Composants React exécutés côté serveur (par défaut)
- **Client Components** : Composants avec interaction utilisateur (avec `'use client'`)

13.2 L'App Router (fichiers et dossiers)

```
app/  
├─ layout.tsx      # Layout racine (HTML, meta, body)  
├─ page.tsx        # Route / (page d'accueil)  
├─ not-found.tsx   # Page 404  
├─ login/  
│   └─ page.tsx    # Route /login  
├─ register/  
│   └─ page.tsx    # Route /register  
├─ profile/  
│   └─ page.tsx    # Route /profile  
└─ movies/  
    └─ [id]/  
        └─ page.tsx # Route /movies/42 (dynamique)
```

13.3 Les layouts

```
// app/layout.tsx – Layout racine (s'applique à TOUTES les pages)  
export default function RootLayout({  
  children,  
}: {  
  children: React.ReactNode;  
}) {  
  return (  

```

```

    <html lang="fr">
      <body className="bg-gray-900 text-white">
        <Navbar />          {/* Barre de navigation sur toutes les pages */}
        <main>{children}</main>  {/* Contenu de la page courante */}
        <Footer />          {/* Pied de page */}
      </body>
    </html>
  );
}

```

13.4 Server Components vs Client Components

```

// SERVER COMPONENT (par défaut) – Aucun "use client"
// - Peut faire des appels API côté serveur
// - Pas de JavaScript envoyé au client
// - Pas de hooks (useState, useEffect)
// - ✅ Pour afficher des données et du contenu statique
async function MovieList() {
  const movies = await fetch("http://api/movies").then(r => r.json());
  return (
    <div>
      {movies.map(m => <div key={m.id}>{m.title}</div>)}
    </div>
  );
}

// CLIENT COMPONENT – Avec "use client"
// - Peut utiliser useState, useEffect, onClick
// - Peut accéder à localStorage, window
// - Le JavaScript est envoyé au client
// - ✅ Pour l'interactivité
"use client";

import { useState } from "react";

function RatingStars() {
  const [rating, setRating] = useState(0);

  return (
    <div onClick={() => setRating(rating + 1)}>
      {rating} étoiles
    </div>
  );
}

```

13.5 Les hooks React essentiels

```

"use client";

import { useState, useEffect, useRef } from "react";
import { useRouter } from "next/navigation";

function MovieSearch() {
  const [query, setQuery] = useState(""); // État local
  const [results, setResults] = useState([]); // Résultats
  const [loading, setLoading] = useState(false); // État de chargement
  const router = useRouter(); // Navigation
  const debounceRef = useRef(null); // Référence (timer)

  useEffect(() => {
    // Effet de bord : appel API quand query change
    if (query.length < 2) return;

    setLoading(true);
    const timer = setTimeout(async () => {
      const data = await searchMovies(query);
      setResults(data);
      setLoading(false);
    }, 300);

    return () => clearTimeout(timer); // Nettoyage
  }, [query]); // Dépendances : se ré-exécute quand query change

  return (
    <div>
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Rechercher..."
      />
      {results.map(m => (
        <div key={m.id} onClick={() => router.push(`/movies/${m.id}`)}>
          {m.title}
        </div>
      ))}
    </div>
  );
}

```

13.6 L'API Axios (services/api.ts)

```

import axios from "axios";

// Configuration de base
const API_URL = process.env.NEXT_PUBLIC_API_URL || "http://localhost:8000";

```

```

const client = axios.create({
  baseURL: API_URL,
  timeout: 10000,
  headers: { "Content-Type": "application/json" }
});

// Intercepteur : ajoute le token JWT à chaque requête
client.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Intercepteur : gère les erreurs globales
client.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem("token"); // Token expiré
    }
    return Promise.reject(error);
  }
);

// API service
export const api = {
  // Authentification
  login: (email: string, password: string) =>
    client.post("/login", { email, password }).then(r => r.data),

  register: (username: string, email: string, password: string) =>
    client.post("/users", { username, email, password }).then(r => r.data),

  // Films
  getMovies: (skip = 0, limit = 20) =>
    client.get(`/movies?skip=${skip}&limit=${limit}`).then(r => r.data),

  getMovie: (id: number) =>
    client.get(`/movies/${id}`).then(r => r.data),

  searchMovies: (query: string) =>
    client.get(`/movies/search?q=${query}&limit=10`).then(r => r.data),

  // Évaluations
  rateMovie: (userId: number, movieId: number, rating: number) =>
    client.post("/ratings", { user_id: userId, movie_id: movieId, rating }).then(r =>
r.data),

  // Recommandations
  getRecommendations: (userId: number, n = 5) =>

```

```

    client.get(`/recommendations?n=${n}`).then(r => r.data),
  });

```

13.7 Gestion des états dans les pages

```

"use client";

import { useState, useEffect } from "react";
import { api, Movie } from "@services/api";

export default function Home() {
  // États
  const [movies, setMovies] = useState<Movie[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  // Chargement initial
  useEffect(() => {
    api.getMovies()
      .then(setMovies)
      .catch(() => setError("Impossible de charger les films"))
      .finally(() => setLoading(false));
  }, []);

  // 3 états possibles pour l'affichage
  if (loading) return <LoadingSkeleton />;
  if (error) return <ErrorMessage message={error} />;
  return <MovieGrid movies={movies} />; // Succès
}

```

13.8 Les pages dynamiques ([id])

```

"use client";

import { useParams } from "next/navigation";
import { useState, useEffect } from "react";
import { api, Movie } from "@services/api";

export default function MovieDetailPage() {
  const params = useParams();
  const movieId = Number(params.id); // Récupère [id] de l'URL

  const [movie, setMovie] = useState<Movie | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    api.getMovie(movieId).then(data => {
      setMovie(data);
    });
  }, [movieId]);
}

```

```

        setLoading(false);
    });
}, [movieId]);

if (loading) return <LoadingSpinner />;
if (!movie) return <NotFound />;

return (
    <div>
        <h1>{movie.title}</h1>
        <p>{movie.genres}</p>
    </div>
);
}

```

15. Tailwind CSS

14.1 Qu'est-ce que Tailwind CSS ?

Tailwind CSS est un framework CSS "utility-first". Au lieu d'écrire du CSS personnalisé, vous utilisez des classes utilitaires pré-définies directement dans votre HTML/JSX.

```

// ❌ CSS traditionnel
// <div class="card">
// .card { background: white; padding: 24px; border-radius: 16px; }

// ✅ Tailwind (tout en classes)
<div className="bg-white p-6 rounded-2xl shadow-lg">

```

14.2 Les classes essentielles par catégorie

Layout

```

<div className="container mx-auto">           // Conteneur centré
<div className="flex">                         // Flexbox
<div className="grid grid-cols-3">           // Grid 3 colonnes
<div className="space-y-4">                 // Espace vertical entre enfants
<div className="items-center justify-between"> // Alignement flex
<div className="gap-6">                     // Espacement grid

```

Espacement (marges et paddings)

```

// Marges : m{t|r|b|l|x|y}-{taille}
// Paddings : p{t|r|b|l|x|y}-{taille}
// Tailles : 0, 1, 2, 3, 4, 5, 6, 8, 10, 12, 16, 20, 24, 32, 40, 48, 56, 64

<div className="m-4">           // margin: 16px

```

```

<div className="p-6">      // padding: 24px
<div className="mx-auto"> // margin: auto horizontal
<div className="py-3">     // padding vertical: 12px
<div className="px-5">     // padding horizontal: 20px
<div className="mt-8">     // margin-top: 32px
<div className="mb-4">     // margin-bottom: 16px
<div className="gap-4">    // gap: 16px (grid/flex)

```

Typographie

```

<p className="text-sm">      // texte petit
<p className="text-lg">      // texte grand
<p className="text-xl">      // encore plus grand
<p className="text-2xl">     // 1.5rem
<p className="text-3xl">...</p> // jusqu'à text-9xl (8rem)

<p className="font-medium">   // gras moyen
<p className="font-bold">    // gras
<p className="font-semibold"> // semi-gras

<p className="text-gray-400"> // gris clair
<p className="text-white">   // blanc
<p className="text-purple-400"> // violet

<p className="text-center">   // centré
<p className="text-left">    // gauche
<p className="truncate">     // coupe avec ... si trop long

```

Couleurs

```

// Texte : text-{couleur}-{nuance}
// Fond : bg-{couleur}-{nuance}
// Bordure : border-{couleur}-{nuance}

// Nuances : 50, 100, 200, 300, 400, 500, 600, 700, 800, 900, 950

// Couleurs disponibles :
gray, red, orange, yellow, green, teal, blue, indigo, purple, pink

// Exemples :
bg-gray-900      // Fond sombre (notre projet)
bg-white/5       // Fond blanc à 5% d'opacité
bg-purple-600/30 // Fond violet à 30%
text-gray-400    // Texte gris
text-white       // Texte blanc
border-white/10  // Bordure blanche à 10%

```

Bordures et coins

```

<div className="rounded-lg">           // border-radius: 8px
<div className="rounded-xl">           // border-radius: 12px
<div className="rounded-2xl">          // border-radius: 16px
<div className="rounded-full">         // cercle parfait

<div className="border border-white/10"> // bordure fine
<div className="border-t border-white/10"> // bordure top seulement

```

Ombres

```

<div className="shadow-sm">           // ombre légère
<div className="shadow-md">           // ombre moyenne
<div className="shadow-lg">           // ombre forte (notre projet)
<div className="shadow-xl">           // ombre très forte
<div className="shadow-2xl">          // ombre max

// Ombres personnalisées avec couleur
hover:shadow-lg hover:shadow-purple-500/25

```

États (hover, focus, etc.)

```

<button className="bg-purple-600 hover:bg-purple-700 transition-colors">
  Cliquez-moi
</button>

<input className="focus:outline-none focus:border-purple-500 focus:ring-2" />

<div className="hover:scale-105 transition-transform duration-300" />

```

Responsive (breakpoints)

```

// sm: 640px, md: 768px, lg: 1024px, xl: 1280px, 2xl: 1536px

// 1 colonne sur mobile, 2 sur tablette, 4 sur desktop
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6" />

// Texte responsive
<h1 className="text-3xl md:text-5xl font-bold" />

// Layout responsive
<div className="flex-col md:flex-row" />

```

Animations

```

<div className="animate-spin" />           // Rotation (loading spinner)
<div className="animate-pulse" />         // Pulsation (skeleton loading)

```

```

<div className="animate-bounce" />           // Rebond
<div className="animate-ping" />             // Effet de notification

// Transitions personnalisées
<div className="transition-all duration-300 ease-in-out" />
<div className="hover:scale-105 transition-transform duration-200" />

```

Arrière-plans et effets

```

// Dégradé (beaucoup utilisé dans notre projet)
<div className="bg-gradient-to-r from-purple-600 to-pink-600" />
<div className="bg-gradient-to-br from-purple-600/30 via-blue-500/20 to-pink-500/30" />

// Flou
<div className="backdrop-blur-lg" />
<div className="backdrop-blur-md" />

// Opacité
<div className="opacity-50 hover:opacity-100" />

```

14.3 Exemple concret (une carte de film)

```

<div className="bg-white/5 backdrop-blur-lg rounded-xl overflow-hidden shadow-lg
  hover:shadow-2xl hover:shadow-purple-500/10
  transition-all duration-300 hover:scale-[1.02] hover:bg-white/10">

  {/* Image/Poster */}
  <div className="h-48 bg-gradient-to-br from-purple-600/30 via-blue-500/20 to-pink-500/30
    flex items-center justify-center">
    <div className="w-16 h-16 bg-gradient-to-br from-purple-500/40 to-pink-500/40
      rounded-2xl flex items-center justify-center text-xl font-bold text-
white/60">
      MR
    </div>
  </div>

  {/* Contenu */}
  <div className="p-4">
    <h3 className="text-white font-semibold truncate">Inception</h3>
    <p className="text-gray-400 text-xs">2010 • Sci-Fi, Action</p>

    {/* Tags genres */}
    <div className="flex gap-1.5 mt-2">
      <span className="px-2 py-0.5 rounded-full text-[10px] bg-cyan-500 text-white">
        Sci-Fi
      </span>
      <span className="px-2 py-0.5 rounded-full text-[10px] bg-red-500 text-white">
        Action
      </span>
    </div>
  </div>

```

```
</div>
</div>
</div>
```

16. Le Système de Recommandation

15.1 Qu'est-ce qu'un système de recommandation ?

Un **système de recommandation** est un algorithme qui prédit ce qu'un utilisateur pourrait aimer, basé sur :

- **L'historique de l'utilisateur** (les films qu'il a notés)
- **Les goûts d'utilisateurs similaires** (filtrage collaboratif)
- **Les caractéristiques des items** (filtrage basé sur le contenu)

15.2 Les approches principales

Approche	Principe	Exemple	Dans notre projet
Filtrage collaboratif	"Les gens qui aiment X aiment aussi Y"	Amazon, Netflix	✅ OUI (basé sur les notes)
Basé sur le contenu	"Ce film ressemble à ceux que tu aimes"	Recommandation par genres	❌ NON
Hybride	Mélange des deux approches	La plupart des systèmes pro	Pas encore

15.3 Le filtrage collaboratif (notre algorithme)

Étape 1 : Créer la matrice utilisateur-film

```
# Données brutes (évaluations)
# userId | movieId | rating
#  1    |    42    |  4.5
#  1    |     7    |  3.0
#  2    |    42    |  5.0
#  2    |    13    |  2.5

# Transformation en matrice pivot
user_movie_matrix = ratings.pivot_table(
    index="userId",      # Lignes = utilisateurs
    columns="movieId",   # Colonnes = films
    values="rating"      # Valeurs = notes
).fillna(0)             # Les films non notés = 0

# Résultat :
# movieId | 7 | 13 | 42 |
# userId 1 | 3.0 | 0.0 | 4.5 |
# userId 2 | 0.0 | 2.5 | 5.0 |
```

Étape 2 : Calculer la similarité entre utilisateurs

```
from sklearn.metrics.pairwise import cosine_similarity
from scipy.sparse import csr_matrix

# La matrice est très sparse (beaucoup de 0)
# On utilise le format CSR pour économiser la mémoire
sparse_matrix = csr_matrix(user_movie_matrix.values)

# Similarité cosinus entre TOUS les utilisateurs
similarity_matrix = cosine_similarity(sparse_matrix)

# Résultat :
# Utilisateur | 1 | 2 | 3 |
# 1 | 1.00 | 0.95 | 0.12 |
# 2 | 0.95 | 1.00 | 0.08 |
# 3 | 0.12 | 0.08 | 1.00 |
# → Les utilisateurs 1 et 2 ont des goûts très similaires (0.95)
# → L'utilisateur 3 a des goûts très différents
```

Étape 3 : Prédire les notes

```
def predict_rating(user_id, movie_id, top_k=10):
    """
    Prédit la note d'un utilisateur pour un film non noté.

    Principe :
    1. Trouver les K utilisateurs les plus similaires à l'utilisateur cible
    2. Regarder comment ils ont noté le film
    3. Faire une moyenne pondérée par leur similarité
    """
    # Index de l'utilisateur
    user_idx = users_ids.index(user_id)

    # Similarités avec tous les autres utilisateurs
    similarities = similarity_matrix[user_idx]

    # Top K utilisateurs similaires (sauf soi-même)
    top_users = np.argsort(similarities)[::-1][1:top_k+1]

    # Notes de ces utilisateurs pour le film
    film_ratings = user_movie_matrix.iloc[top_users, movie_idx]

    # Filtrer ceux qui ont effectivement noté
    valid = film_ratings > 0
    if valid.sum() < 2:
        return None # Pas assez de données

    # Moyenne pondérée par la similarité
    weights = similarities[top_users][valid]
```

```

ratings = film_ratings[valid]

return np.average(ratings, weights=weights)

```

Étape 4 : Générer les recommandations

```

def recommend(user_id, n=5):
    """
    Génère N recommandations pour un utilisateur.

    1. Pour chaque film NON noté par l'utilisateur
    2. Prédire la note
    3. Retourner les N films avec les meilleures prédictions
    """
    predictions = []

    for movie_id in all_movies:
        if movie_id not in userRatedMovies:
            predicted = predict_rating(user_id, movie_id)
            if predicted:
                predictions.append({
                    "movieId": movie_id,
                    "predicted_rating": predicted
                })

    # Trier par note prédite décroissante
    predictions.sort(key=lambda x: x["predicted_rating"], reverse=True)

    return predictions[:n]

```

15.4 Visualisation du processus

Données brutes (MovieLens) :

userId	movieId	rating
1	42	4.5
1	7	3.0
2	42	5.0
2	13	2.5
3	7	4.0



Matrice utilisateur-film (pivot_table) :

userId	7	13	42
1	3.0	0	4.5

2	0	2.5	5.0
3	4.0	0	0

↓



Similarité cosinus :

	1	2	3
1	1.00	0.95	0.85
2	0.95	1.00	0.50
3	0.85	0.50	1.00

↓



Prédiction pour User 3, Film 42 :

- Voisins similaires : User 1 (0.85), User 2 (0.50)
- Leurs notes pour film 42 : 4.5, 5.0
- Prédiction : $(4.5 \times 0.85 + 5.0 \times 0.50) / (0.85 + 0.50) = 4.69$

↓



Recommandations finales pour User 3 :

1. Film 42 (prédit : 4.69 ★)
2. Film 13 (prédit : 3.50 ★)
3. ...

15.5 Les notebooks Jupyter

Le projet inclut 6 notebooks qui couvrent le cycle complet de Data Science :

01_importation.ipynb — Import des données

```
# Chargement des datasets MovieLens
movies = pd.read_csv("data/ml-latest-small/movies.csv")
ratings = pd.read_csv("data/ml-latest-small/ratings.csv")

print(f"Films : {len(movies)}")
print(f"Évaluations : {len(ratings)}")
print(f"Utilisateurs : {ratings['userId'].nunique()}")
```

02_nettoyage.ipynb — Nettoyage

```
# Valeurs manquantes, doublons, anomalies
print(ratings.isnull().sum())
print(ratings['rating'].describe())

# Suppression des doublons
ratings = ratings.drop_duplicates()
```

```
# Vérification des plages de notes
assert ratings['rating'].between(0.5, 5.0).all()
```

03_analyse.ipynb — Analyse exploratoire

```
# Distribution des notes
ratings['rating'].hist(bins=10)

# Top 10 des films les mieux notés
top_movies = ratings.groupby('movieId')['rating'].agg(['mean', 'count'])
top_movies[top_movies['count'] > 10].sort_values('mean', ascending=False)
```

04_preparation.ipynb — Préparation

```
# Création de la matrice utilisateur-film
user_movie_matrix = ratings.pivot_table(
    index='userId', columns='movieId', values='rating'
).fillna(0)

print(f"Matrice : {user_movie_matrix.shape}")
print(f"Sparsité : {1 - (ratings.shape[0] / (user_movie_matrix.shape[0] *
user_movie_matrix.shape[1])):.2%}")
```

05_modelisation.ipynb — Modélisation (le cœur)

```
# Construction et entraînement du modèle
from backend.recommender import MovieRecommender

recommender = MovieRecommender()
recommender.fit(ratings, movies)

# Test des recommandations
recommendations = recommender.recommend(user_id=1, n_recommendations=5)

# Sauvegarde
recommender.save("models/recommendation_model.pkl")
```

06_evaluation.ipynb — Évaluation

```
# Évaluer la qualité des prédictions
# RMSE, MAE, précision@k, rappel@k
```

15.6 Améliorations possibles

```
# 1. Filtrage basé sur le contenu (genres)
def content_based_recommendations(movie_id, n=5):
    target = movies[movies['movieId'] == movie_id]
    target_genres = set(target['genres'].split('|'))

    scores = movies.apply(lambda row:
        len(set(row['genres'].split('|')) & target_genres), axis=1)

    return movies.iloc[np.argsort(scores)[::-1][1:n+1]]

# 2. SVD (Singular Value Decomposition) – Plus performant
from scipy.sparse.linalg import svds

U, sigma, Vt = svds(sparse_matrix, k=50)
predicted_ratings = U @ np.diag(sigma) @ Vt

# 3. Pondération par la popularité
popularity_score = np.log1p(movie_rating_counts)
weighted_rating = predicted_rating * (1 + 0.1 * popularity_score)
```

17. Guide de Déploiement

16.1 Prérequis

```
# 1. Installer Docker Desktop
# Aller sur https://www.docker.com/products/docker-desktop/

# 2. Télécharger les données MovieLens
# https://grouplens.org/datasets/movielens/latest/
# Extraire dans data/ml-latest-small/

# 3. Installer Python 3.11+
# https://www.python.org/downloads/

# 4. Installer Node.js 20+
# https://nodejs.org/
```

16.2 Configuration initiale

```
# 1. Cloner ou créer le projet
cd DATA-SCIENCE

# 2. Créer le fichier .env
echo "JWT_SECRET_KEY=generer-une-cle-securisee" > .env
# openssl rand -hex 32
```

```
# 3. Démarrer avec Docker
docker-compose up --build

# 4. Dans un autre terminal, charger les données
docker-compose exec backend python seed_data.py
```

16.3 Développement sans Docker

```
# TERMINAL 1 : Base de données
docker run --name movie-db \
  -e POSTGRES_DB=movie_recommender \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_PASSWORD=postgres \
  -p 5432:5432 \
  -d postgres:15-alpine

# TERMINAL 2 : Backend
cd backend
python -m venv venv
source venv/bin/activate # ou venv\Scripts\activate sur Windows
pip install -r requirements.txt
python seed_data.py
uvicorn main:app --reload

# TERMINAL 3 : Frontend
cd frontend
npm install
npm run dev
```

16.4 Prochaines étapes (améliorations)

Tests

```
# Backend
pip install pytest pytest-cov
pytest tests/ --cov=backend

# Frontend
npm test
```

CI/CD (GitHub Actions)

```
# .github/workflows/ci.yml
name: CI
on: [push]

jobs:
  test:
```

```

runs-on: ubuntu-latest
services:
  postgres:
    image: postgres:15
    env:
      POSTGRES_DB: test_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports: ["5432:5432"]

steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-python@v5
    with: { python-version: "3.11" }

  - name: Install dependencies
    run: pip install -r backend/requirements.txt

  - name: Run tests
    run: pytest backend/tests/

```

Monitoring

```

# Ajouter des logs structurés
import logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

@app.get("/movies")
def get_movies(...):
    logger.info(f"GET /movies - skip={skip}, limit={limit}")
    ...

```

18. Ressources

17.1 Documentation officielle

Technologie	Documentation	Pourquoi la lire
Python	La base	Pour tout comprendre
NumPy	Guide du débutant	Pour maîtriser les arrays
Pandas	User Guide	Pour la manipulation de données
Scikit-learn	Getting Started	Pour le ML
SciPy	Sparse matrices	Pour les matrices creuses
FastAPI	Tutorial	Pour l'API

SQLAlchemy	ORM Tutorial	Pour la base de données
Pydantic	Models	Pour la validation
PostgreSQL	Manual	Pour la BDD
Docker	Get Started	Pour la conteneurisation
Next.js	App Router	Pour le frontend
Tailwind CSS	Utility-First	Pour le design

17.2 Livres recommandés

1. **"Python pour la Data Science"** — Aurélien Géron (le meilleur pour commencer)
2. **"Python for Data Analysis"** — Wes McKinney (le créateur de Pandas)
3. **"Hands-On Machine Learning"** — Aurélien Géron (la bible du ML)
4. **"FastAPI - Modern Python Web Development"** — Bill Lubanovic

17.3 Plateformes d'apprentissage

1. **Kaggle** (kaggle.com) — Datasets, compétitions, notebooks gratuits
2. **DataCamp** (datacamp.com) — Cours interactifs de Data Science
3. **OpenClassrooms** (openclassrooms.com) — Cours en français
4. **Le Wagon** (lewagon.com) — Bootcamp Data Science

17.4 Projets similaires pour s'inspirer

1. Système de recommandation de musique (Spotify)
2. Système de recommandation d'articles (Medium)
3. Système de recommandation de produits (Amazon)
4. Filtrage collaboratif pour séries TV (Netflix)

17.5 Commandes utiles (aide-mémoire)

```
# Python
python -m venv venv          # Créer un environnement virtuel
source venv/bin/activate     # Activer (Mac/Linux)
venv\Scripts\activate        # Activer (Windows)
pip install -r requirements.txt # Installer les dépendances
pip freeze > requirements.txt  # Mettre à jour les dépendances

# Docker
docker-compose up -d          # Démarrer les services
docker-compose logs -f backend # Voir les logs
docker-compose exec backend bash # Terminal dans le conteneur
docker-compose down           # Arrêter les services
docker-compose down -v        # Arrêter et supprimer les volumes

# Git
git status                    # Voir l'état
git add .                     # Stager les changements
```

```
git commit -m "message"      # Commiter
git push                      # Pousser
git pull                      # Tirer

# PostgreSQL
psql -U postgres -d movie_recommender # Connexion
\l                             # Liste des BDD
\dt                           # Liste des tables
\d movies                      # Structure d'une table
```

Conclusion

Félicitations ! 🎉 Vous avez maintenant un guide complet pour comprendre, utiliser et améliorer ce système de recommandation de films.

Rappelez-vous les concepts clés :

1. **La Data Science est un processus itératif** — Analyse → Nettoie → Modélise → Évalue → Recommence
2. **Maîtrisez les fondamentaux** — Pandas, NumPy, Scikit-learn sont les piliers
3. **L'architecture compte** — Une bonne séparation backend/frontend/ML facilite tout
4. **Testez, testez, testez** — Sans tests, vous ne savez pas si ça marche
5. **Documentez** — Votre futur vous remerciera

Les 3 choses à retenir absolument :

1. `ratings.pivot_table(index="userId", columns="movieId", values="rating")` — Pour créer la matrice utilisateur-film
2. `cosine_similarity(sparse_matrix)` — Pour calculer les similarités
3. `np.average(ratings, weights=similarities)` — Pour prédire les notes

Le mot de la fin : Le meilleur moyen d'apprendre est de faire. Lancez le projet, cassez des choses, et amusez-vous !

